UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Fortalecendo a Segurança de Software: Estudo de Caso de uma Pipeline com SBOM, SAST e DAST no POP-ERP

PEDRO VITOR BEZERRA CLEMENTE

Orientador: Prof. Dr. Eduardo Lucena Falcão

Natal, RN, 9 de dezembro de 2025

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Fortalecendo a Segurança de Software: Estudo de Caso de uma Pipeline com SBOM, SAST e DAST no POP-ERP

PEDRO VITOR BEZERRA CLEMENTE

Orientador: Prof. Dr. Eduardo Lucena Falcão

Trabalho de Conclusão de Curso de Graduação na modalidade Monografia, submetido como parte dos requisitos necessários para conclusão do curso de Engenharia de Computação pela Universidade Federal do Rio Grande do Norte (UFRN/CT).

Natal, RN, 9 de dezembro de 2025

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI

Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Clemente, Pedro Vitor Bezerra.
Fortalecendo a segurança de software: estudo de caso de uma pipeline com SBOM, SAST e DAST no POP-ERP / Pedro Vitor Bezerra Clemente. – 2025.
67 f.: il.

Trabalho de Conclusão de Curso – TCC (graduação) –
Universidade Federal do Rio Grande do Norte, Centro de Tecnologia, Curso de Engenharia da Computação, Natal, RN, 2025.
Orientação: Prof. Dr. Eduardo de Lucena Falcão.

1. Pipeline de Segurança de Software – TCC. 2. Práticas de Desenvolvimento, Segurança e Operações – TCC. 3. Análise de Cadeia de Suprimentos – TCC. 4. Teste de Segurança de Aplicação Estática (SAST) – TCC. 5. Teste de Segurança de Aplicação Dinâmica (DAST) – TCC. I. Falcão, Eduardo de Lucena. II. Título.

RN/UF/BCZM CDU 004.72

Elaborado por Fernanda de Medeiros Ferreira Aquino – CRB - 15/301

Fortalecendo a Segurança de Software: Estudo de Caso de uma Pipeline com SBOM, SAST e DAST no POP-ERP

Pedro Vitor Bezerra Clemente

Monografia aprovada em 3 de dezembro de 2025, pela banca examinadora composta pelos seguintes membros:

Prof. Dr. Eduardo de Lucena Falcão (orientador) . . . . . DCA/UFRN

Prof. Dr. Carlos Manuel Dias Viegas . . . . . DCA/UFRN

Profª Drª Taniro Chacon Rodrigues . . . . . EAJ/UFRN

Agradecimentos

Ao meu pai, cuja partida em 2023 deixou uma saudade imensa, mas cuja memória continua sendo minha maior fonte de inspiração. Seus valores, sua dedicação e seu amor permanecerão sempre como guia em cada conquista da minha vida. Este trabalho é também em sua homenagem. À minha mãe, pelo apoio incondicional, pela força em todos os momentos e por nunca deixar faltar motivação para que eu chegasse até aqui. Ao meu irmão e à minha irmã, pelo incentivo constante e pela presença que sempre tornou a jornada mais leve. Ao professor Eduardo Lucena, meu orientador, agradeço pela orientação precisa e pelo comprometimento que contribuíram diretamente para a qualidade deste trabalho. Aos meus amigos, pela companhia, apoio e compreensão nos momentos difíceis e pelas alegrias compartilhadas durante esta fase. À equipe do PoP-RN, onde atuo como bolsista, pelo ambiente de aprendizado e pela oportunidade de crescimento profissional que influenciou de forma significativa o desenvolvimento deste trabalho.

A todos que contribuíram direta ou indiretamente para a realização deste TCC, deixo o meu sincero agradecimento.

Resumo

Este trabalho apresenta o desenvolvimento e a implementação de uma pipeline de segurança automatizada integrada ao processo de Integração e Entrega Contínuas (CI/CD) do sistema PoP-ERP, com o objetivo de fortalecer o desenvolvimento seguro e mitigar vulnerabilidades ao longo de todo o ciclo de vida do software. A solução adota princípios de DevSecOps e combina três camadas complementares de verificação: análise estática do código-fonte (SAST, abreviação para o inglês *Static Application Security Testing*), análise da cadeia de suprimentos de software e testes dinâmicos de segurança (DAST, do inglês *Dynamic Application Security Testing*). A metodologia empregada compreende levantamento bibliográfico, modelagem arquitetural, implementação incremental, execução experimental e avaliação crítica dos resultados obtidos. Os experimentos demonstraram a efetividade da pipeline na identificação de vulnerabilidades estruturais, falhas em dependências e comportamentos inseguros em tempo de execução, evidenciando a importância da automação e da integração de segurança no processo de desenvolvimento. Como contribuição, o trabalho estabelece uma abordagem reprodutível e aplicável a outros ambientes CI/CD, reforçando a maturidade em segurança do PoP-ERP e promovendo a consolidação de práticas DevSecOps na instituição.

Palavras-chave: DevSecOps; CI/CD; SAST; DAST; cadeia de suprimentos de software; PoP-ERP.

Abstract

This work presents the development and implementation of an automated security pipeline integrated into the Continuous Integration and Continuous Delivery (CI/CD) process of the PoP-ERP system, aiming to strengthen secure development and mitigate vulnerabilities throughout the software lifecycle. The solution adopts DevSecOps principles and combines three complementary layers of verification: static application security testing (SAST), software supply chain analysis, and dynamic application security testing (DAST). The methodology includes a systematic literature review, architectural modeling, incremental implementation, controlled experimentation, and critical evaluation of the obtained results. The experiments demonstrated the effectiveness of the pipeline in identifying structural vulnerabilities, dependency flaws, and insecure runtime behaviors, highlighting the importance of automation and continuous security integration within the development process. As a contribution, the study provides a reproducible approach applicable to other CI/CD environments, strengthens the security posture of PoP-ERP, and promotes the consolidation of DevSecOps practices within the institution.

Keywords: DevSecOps; CI/CD; SAST; DAST; Software Supply Chain; PoP-ERP.

Sumário

Sumário i

Lista de Figuras iii

Lista de Tabelas v

1 Introdução 1
- 1.1 Contexto e motivação de trabalho . . . 2
  - 1.1.1 Incidentes de segurança com sistemas ERP . . . 3
- 1.2 Objetivo . . . 3
- 1.3 Contribuições . . . 4
- 1.4 Estrutura do Trabalho . . . 4

2 Fundamentação Teórica 7
- 2.1 Vulnerabilidades de Software: Conceitos e Formas de Classificação . . . 7
  - 2.1.1 O que é uma vulnerabilidade? . . . 7
  - 2.1.2 Classificações de vulnerabilidades . . . 8
- 2.2 Teste Estático de Segurança de Aplicações . . . 8
- 2.3 Análise da cadeia de suprimentos de software . . . 9
- 2.4 Teste Dinâmico de Segurança de Aplicações . . . 10
- 2.5 Exemplos de vulnerabilidades detectadas por SAST, SBOM e DAST . . . 11
- 2.6 Tecnologias que compõem o sistema PoP-ERP . . . 11
  - 2.6.1 Linguagem de Programação Python . . . 11
  - 2.6.2 Framework Django . . . 12
  - 2.6.3 Banco de Dados PostgreSQL . . . 12

3 Revisão de literatura 13
- 3.1 Estudo precursor . . . 13
- 3.2 Metodologia de pesquisa . . . 14
- 3.3 Artigos relevantes ao tema . . . 15
- 3.4 Produção científica sobre o tema . . . 17
- 3.5 Metanálise dos Resultados da Revisão Bibliográfica . . . 18

4 Solução Proposta 19
- 4.1 Metodologia . . . 19
- 4.2 Arquitetura . . . 22

4.2.1 Análise Estática de Código com Bandit 23
4.2.2 Análise de Vulnerabilidades com Trivy 23
4.2.3 Análise Dinâmica com OWASP ZAP 24
4.3 Integração ao pipeline de CI/CD 25
4.3.1 Fase 1: Integração do Bandit (SAST) 26
4.3.2 Fase 2: Integração do Trivy para análise de dependências 26
4.3.3 Fase 3: Integração do OWASP ZAP (DAST) 27
4.3.4 Pipeline final integrada 28
4.3.5 Conclusão da integração 29

5 Avaliação do Pipeline de Segurança 31
5.1 Descrição do Ambiente Experimental 31
5.1.1 Configurações 31
5.2 Avaliação da Análise Estática (SAST) 32
5.2.1 Resultados Gerais do SAST 32
5.2.2 Discussão e Ações Corretivas no SAST 32
5.3 Avaliação da Análise de SBOM (Trivy) 33
5.3.1 Resultados da Análise SBOM 34
5.3.2 Discussão e Ações Corretivas SBOM 34
5.4 Avaliação do Teste Dinâmico (DAST) 35
5.4.1 Resultados Obtidos 35
5.4.2 Discussão e Ações Corretivas DAST 35
5.4.3 Síntese Geral da Ferramenta OWASP ZAP 37
5.5 Síntese Integrada da Avaliação 37

6 Conclusão 39
6.1 Síntese dos Resultados 39
6.2 Dificuldades Técnicas e Organizacionais 39
6.3 Impacto na Cultura de Desenvolvimento 40
6.4 Trabalhos Futuros 40
6.5 Considerações Finais 41

Referências bibliográficas 42

A Informações adicionais 49

Lista de Figuras

4.1 Pipeline inicial do PoP-ERP antes da integração das etapas de segurança . 25
4.2 Pipeline na fase 1 após a adição do Bandit 26
4.3 Pipeline na fase 2 após a adição do Trivy 27
4.4 Pipeline na fase 3 após adição do OWASP ZAP 28
4.5 Pipeline versão final 29

5.1 Resultados do relatório completo gerado pelo Bandit. 33

Lista de Tabelas

3.1 Resumo dos principais estudos selecionados na revisão bibliográfica . . . 15

4.1 Etapas cronológicas da metodologia de implementação da pipeline de segurança 21

5.1 Vulnerabilidades classificadas como *High severity* e *High confidence* identificadas pelo Bandit 32
5.3 Vulnerabilidades encontradas na análise SBOM 34
5.4 Resumo das vulnerabilidades detectadas pelo OWASP ZAP (Baseline) . . 35
5.5 Comparativo entre SAST, SBOM e DAST 37

A.1 Tabela completa da Pesquisa Bibliográfica 49

Capítulo 1

Introdução

Um dos principais pilares da sociedade moderna é a atuação dos provedores de serviços de internet, conhecidos como *Internet Service Providers* (ISPs). Essas organizações têm como propósito fundamental conectar usuários por meio de diferentes tecnologias — como fibra óptica, cabos, redes sem fio, entre outras —, oferecendo não apenas o acesso à internet, mas também uma variedade de serviços complementares, como hospedagem de sites, provedores de e-mail e soluções de segurança de rede. Dessa forma, os ISPs desempenham um papel essencial na infraestrutura digital contemporânea, sustentando tanto as atividades cotidianas da população quanto o funcionamento de instituições públicas e privadas.

Nesse contexto, destaca-se o papel do Ponto de Presença da Rede Nacional de Ensino e Pesquisa no Rio Grande do Norte (PoP-RN), localizado na Universidade Federal do Rio Grande do Norte (UFRN), na cidade de Natal. A instituição exerce uma função estratégica ao prover conectividade e serviços de rede voltados principalmente a órgãos governamentais, instituições de ensino, pesquisa e infraestrutura crítica, como hospitais e delegacias. Desde sua criação, em 1996, por meio de um acordo de cooperação entre a Rede Nacional de Ensino e Pesquisa (RNP) e a UFRN, o PoP-RN vem expandindo sua atuação, consolidando uma rede de alta capacidade que abrange tanto a capital quanto diversos municípios do interior do Estado.

A gestão do PoP-RN é complexa e distribuída entre múltiplos setores que atuam de maneira integrada para garantir a qualidade e a continuidade dos serviços prestados. Entre esses setores, destacam-se: administração, serviços, segurança, operações, infraestrutura e desenvolvimento. A área administrativa é responsável pela gestão contratual e jurídica; os setores de serviços e operações, por meio do Centro de Operações de Rede, realizam o monitoramento constante de centenas de equipamentos e enlaces de rede. O setor de segurança atua na prevenção e mitigação de incidentes por meio do Sistema de Gestão de Incidentes de Segurança da RNP, enquanto o setor de infraestrutura é encarregado da instalação e manutenção dos equipamentos físicos em campo. Já o setor de desenvolvimento dedica-se à criação e manutenção de soluções tecnológicas, entre elas, o PoP-ERP, um sistema de *Enterprise Resource Planning* (ERP) desenvolvido internamente com o objetivo de automatizar e otimizar os processos internos dos demais setores.

Os sistemas ERP, como o PoP-ERP, são projetados para integrar e centralizar o gerenciamento de processos organizacionais em uma única plataforma, abrangendo áreas como

finanças, recursos humanos, operações, estoque e cadeia de suprimentos. Conforme Laudon e Laudon (2020), "os sistemas ERP integram processos de negócios em uma única plataforma, permitindo que as empresas automatizem tarefas e melhorem a visibilidade dos dados". Esses sistemas proporcionam comunicação em tempo real entre diferentes setores, eliminam redundâncias e aumentam a eficiência administrativa (O'BRIEN; MARAKAS, 2011). No contexto do PoP-RN, o PoP-ERP tem papel central na sustentação tecnológica das operações internas, atuando como ferramenta estratégica para o controle e a execução de atividades essenciais. Contudo, a crescente dependência de soluções digitais como o PoP-ERP também impõe novos desafios relacionados à segurança de software, exigindo a adoção de práticas e ferramentas que assegurem a integridade, a confidencialidade e a disponibilidade das informações. Diante disso, este trabalho propõe o desenvolvimento e a aplicação de uma *pipeline* de segurança automatizada, voltada à detecção e mitigação de vulnerabilidades no ciclo de vida do PoP-ERP, fortalecendo as práticas de *DevSecOps* e contribuindo para a maturidade em segurança da informação dentro do PoP-RN.

1.1 Contexto e motivação de trabalho

O projeto PoP-ERP encontra-se em um estágio avançado de desenvolvimento, com a maioria das funcionalidades originalmente planejadas já implementadas e em operação. Atualmente, o sistema passa por um processo contínuo de manutenção evolutiva, focado na correção de falhas menores e na adição de novas funcionalidades de baixo impacto. Durante as fases iniciais do desenvolvimento, o principal objetivo foi disponibilizar rapidamente um *Minimum Viable Product* (MVP), priorizando a entrega funcional do sistema. Entretanto, essa abordagem resultou em uma atenção limitada aos aspectos de segurança de software, criando uma lacuna relevante a ser abordada.

Com o intuito de suprir essa deficiência e elevar o nível de maturidade em segurança do PoP-ERP, o presente trabalho propõe a implementação de uma *pipeline* de segurança automatizada integrada ao processo de *Continuous Integration* e *Continuous Delivery* (CI/CD). Essa *pipeline* tem como objetivo principal incorporar mecanismos de análise estática (*Static Application Security Testing* — SAST), análise dinâmica (*Dynamic Application Security Testing* — DAST) e gestão da cadeia de suprimentos de software por meio da geração e análise de um *Software Bill of Materials* (SBOM). A etapa de SAST permitirá inspecionar o código-fonte em busca de vulnerabilidades e práticas de programação inseguras; a análise de SBOM possibilitará o mapeamento detalhado de dependências externas e a detecção de componentes vulneráveis ou desatualizados; e a etapa de DAST realizará varreduras ativas e passivas sobre o sistema em execução, simulando ataques reais para identificar falhas exploráveis em ambiente de produção ou *staging*.

A integração dessas etapas dentro da *pipeline* visa estabelecer um ciclo contínuo de detecção, prevenção e mitigação de vulnerabilidades, garantindo que cada nova atualização do PoP-ERP passe por verificações de segurança automatizadas antes de ser implantada. Dessa forma, busca-se não apenas fortalecer a robustez, a confiabilidade e a conformidade do sistema, mas também consolidar uma cultura de *DevSecOps* no âmbito do PoP-RN, promovendo o desenvolvimento seguro como parte integrante do processo de

entrega de software.

1.1.1 Incidentes de segurança com sistemas ERP

A constante disputa entre desenvolvedores de software e agentes maliciosos configura um cenário dinâmico e ininterrupto no campo da cibersegurança. Grupos criminosos, especialmente aqueles voltados a ataques de *ransomware*, têm explorado vulnerabilidades em aplicações para obter acesso não autorizado a dados sensíveis, interromper operações críticas e extorquir organizações públicas e privadas mediante demandas financeiras de alto valor. Nos últimos anos, a ocorrência de incidentes envolvendo sistemas corporativos complexos, como plataformas ERP, tem evidenciado a gravidade dessas ameaças. Um caso notório é o da vulnerabilidade CVE-2025-61882 no Oracle *E-Business Suite*, amplamente explorada por grupo de ransomware Cl0p (CORPORATION, 2025; UKHANOV et al., 2025; INTELLIGENCE, 2025). Esses eventos reforçam a urgência de incorporar práticas de segurança desde as etapas iniciais do ciclo de desenvolvimento de software (*Software Development Life Cycle* — SDLC), assegurando uma postura proativa de proteção e mitigação de riscos que afetam diretamente a continuidade operacional e a estabilidade financeira das organizações.

Em outubro de 2025, foi divulgado um alerta de segurança pela Oracle relativo à vulnerabilidade crítica CVE-2025-61882 no Oracle E-Business Suite (EBS), que permite execução remota de código sem autenticação. (CORPORATION, 2025) Essa falha afeta especificamente o componente de *Concurrent Processing* integrado ao BI Publisher nas versões 12.2.3 a 12.2.14 (CYBER, 2025). Segundo relatórios de inteligência, o grupo de extorsão Cl0p explorou essa vulnerabilidade em uma campanha de e-mails para executivos, alegando ter exfiltrado dados de ambientes EBS vulneráveis (UKHANOV et al., 2025). A Oracle lançou um patch emergencial em 4 de outubro, fornecendo também indicadores de comprometimento (IOCs) para detecção (CORPORATION, 2025). Agências de segurança como a CrowdStrike confirmaram que o exploit zero-day foi realmente usado na campanha de extorsão, atribuindo parte das atividades ao grupo Gracious Spider, mas sem descartar outros atores (INTELLIGENCE, 2025).

Esse cenário evidencia que as violações de segurança em sistemas empresariais são eventos recorrentes e de alto impacto, uma vez que tais plataformas se tornaram alvos estratégicos de grupos cibernéticos maliciosos. Organizações que operam com grandes volumes de dados e fluxos financeiros elevados figuram entre os principais alvos desses ataques, motivados pelo potencial de obtenção de ganhos ilícitos. As ameaças incluem o vazamento de informações sensíveis de clientes, a venda de dados sigilosos a concorrentes e a interrupção deliberada de operações críticas, resultando em prejuízos diários expressivos.

1.2 Objetivo

O objetivo geral deste trabalho é o desenvolvimento e implementação de uma *pipeline* de segurança automatizada voltada ao fortalecimento do processo de desenvolvimento seguro e ao monitoramento contínuo da segurança de aplicações web, com foco no sistema

PoP-ERP. A proposta visa integrar práticas de *DevSecOps* ao ciclo de vida de desenvolvimento de software (SDLC), promovendo a automação das etapas de análise de vulnerabilidades, o reforço das boas práticas de codificação segura e a detecção precoce de falhas que possam comprometer a integridade, a confidencialidade ou a disponibilidade das aplicações.

A *pipeline* será estruturada de modo a incorporar diferentes camadas de verificação de segurança, contemplando análise estática (SAST), análise dinâmica (DAST) e análise da cadeia de suprimentos de software por meio da geração e inspeção do SBOM. Com isso, busca-se estabelecer um ciclo contínuo de avaliação e correção de vulnerabilidades, integrando o controle de qualidade de segurança ao processo de integração e entrega contínua (CI/CD) do sistema.

Ao término da implementação, espera-se que o PoP-ERP atinja um nível superior de resiliência cibernética, caracterizado pela presença de mecanismos automatizados de monitoramento e pela consolidação de uma cultura *DevSecOps* entre os desenvolvedores. Essa cultura deverá estimular a identificação, triagem e correção sistemática de vulnerabilidades ao longo de todo o ciclo de vida do software, assegurando a conformidade com padrões modernos de segurança e contribuindo para a sustentabilidade operacional e a confiabilidade do ambiente computacional do PoP-RN.

1.3 Contribuições

As contribuições deste trabalho estão organizadas em dois eixos centrais e complementares. O primeiro eixo consiste no desenvolvimento de uma metodologia formal, sistemática e replicável para a incorporação de mecanismos de segurança em pipelines de CI/CD, abrangendo etapas de análise estática, análise da cadeia de suprimentos e testes dinâmicos, com o objetivo de identificar vulnerabilidades e inconsistências de configuração ao longo de todo o ciclo de desenvolvimento. O segundo eixo refere-se à aplicação prática dessa metodologia no contexto do sistema PoP-ERP, demonstrando sua viabilidade operacional por meio da detecção de vulnerabilidades reais, da implementação de correções orientadas pelas evidências produzidas pelas ferramentas e do consequente fortalecimento da postura de segurança da plataforma. Juntos, esses eixos evidenciam tanto a robustez da abordagem proposta quanto sua capacidade de promover melhorias tangíveis na segurança de sistemas institucionais.

1.4 Estrutura do Trabalho

Este trabalho está organizado da seguinte forma:

- **Capítulo 2** — Apresenta a fundamentação teórica, abordando conceitos essenciais de vulnerabilidades de software, classificações e as técnicas de segurança (SAST, SBOM e DAST) utilizadas na pipeline.
- **Capítulo 3** — Promove uma revisão de literatura sobre o tema que será abordado no trabalho, levantando artigos e dados estatísticos relevantes ao assunto geral.

- **Capítulo 4** — Descreve a metodologia proposta, detalhando o desenho, implementação e integração da pipeline de segurança.
- **Capítulo 5** — Apresenta os resultados obtidos com a aplicação da metodologia ao PoP-ERP, bem como a análise crítica dos achados.
- **Capítulo 6** — Conclui o trabalho apresentando suas conquistas, dificuldades enfrentadas bem como a satisfação com o resultado final da pipeline e finaliza apresentando desafios futuros que podem ser explorados em novos trabalhos.

Capítulo 2

Fundamentação Teórica

Este capítulo apresenta a fundamentação teórica que orienta o desenvolvimento da pipeline de segurança proposta neste trabalho. Inicialmente, descrevem-se os conceitos fundamentais relacionados a vulnerabilidades de software e aos principais sistemas de classificação adotados pela indústria, fornecendo a base necessária para compreender a natureza dos riscos envolvidos. Em seguida, são introduzidas as três abordagens de segurança utilizadas — SAST, SBOM e DAST — incluindo suas características, limitações e aplicações práticas no ciclo de desenvolvimento seguro. Posteriormente, discutem-se as tecnologias selecionadas para compor o pipeline, tais como Bandit, Trivy e OWASP ZAP, justificando sua escolha a partir de critérios técnicos e de integração. Por fim, apresenta-se a arquitetura tecnológica do sistema PoP-ERP, destacando as particularidades que influenciam sua análise de segurança e sua integração ao modelo DevSecOps defendido neste estudo.

2.1 Vulnerabilidades de Software: Conceitos e Formas de Classificação

A segurança de software fundamenta-se na identificação, análise e mitigação de vulnerabilidades, entendidas como falhas ou fraquezas que podem comprometer a integridade, a confidencialidade ou a disponibilidade de um sistema. A compreensão aprofundada desse conceito e dos mecanismos de classificação existentes é essencial, pois orienta a utilização das três abordagens de segurança adotadas neste estudo — SAST, SBOM e DAST — e evidencia a necessidade de técnicas complementares ao longo do ciclo de desenvolvimento seguro.

2.1.1 O que é uma vulnerabilidade?

Uma vulnerabilidade pode surgir de diversos fatores, como erros de implementação, falhas de configuração, dependências desatualizadas, interações inadequadas entre componentes ou uso incorreto de bibliotecas e APIs. Essas condições inadequadas possibilitam que um agente malicioso altere o comportamento esperado do software, podendo resultar em impactos severos, como vazamento de dados sensíveis, execução remota de

comandos, elevação de privilégios, negação de serviço ou acesso não autorizado. Assim, o tratamento de vulnerabilidades exige uma visão abrangente que considere tanto aspectos internos ao código quanto elementos externos relacionados ao ambiente e às dependências do sistema.

2.1.2 Classificações de vulnerabilidades

Na indústria de software, são utilizados sistemas padronizados que permitem organizar e priorizar vulnerabilidades conforme sua natureza e impacto. Entre esses sistemas, destaca-se o *Common Weakness Enumeration* (CWE), que cataloga fraquezas estruturais responsáveis por originar vulnerabilidades. Outro mecanismo amplamente utilizado é o *Common Vulnerabilities and Exposures* (CVE), que atribui um identificador único a vulnerabilidades conhecidas publicamente em softwares, dependências e bibliotecas.

Além disso, a severidade de cada vulnerabilidade costuma ser avaliada por meio do *Common Vulnerability Scoring System* (CVSS), que considera fatores como impacto sobre confidencialidade, integridade e disponibilidade, complexidade do ataque, vetor de exploração, existência de exploits públicos e disponibilidade de correções. A escala utilizada varia entre 0.0 e 10.0, sendo:

1. baixa, de 0,0 a 3,9;
2. média, de 4,0 a 6,9;
3. alta, de 7,0 a 8,9;
4. crítica, de 9,0 a 10,0.

Essa classificação padronizada permite a priorização objetiva de ações corretivas e facilita a comparação entre diferentes tipos de vulnerabilidades.

2.2 Teste Estático de Segurança de Aplicações

A análise de segurança de aplicações por meio de técnicas estáticas (SAST) consiste em examinar o código-fonte ou binário de um software sem executá-lo, com o objetivo de identificar vulnerabilidades (por exemplo: uso de funções inseguras, fluxos de dados não validados, credenciais codificadas) nas fases iniciais do ciclo de vida de desenvolvimento (SIAVVAS et al., 2018). Essa abordagem "shift-left" permite feedback precoce aos desenvolvedores, reduzindo o custo de correção de falhas e mitigando riscos antes da execução ou do deploy (SHEN et al., 2023). Apesar dos seus benefícios, a adoção da SAST enfrenta barreiras como falsos positivos, dificuldade de integração no fluxo de CI/CD e resistência de equipes de desenvolvimento (WADHAMS; IZURIETA; REINHOLD, 2024). Portanto, embora as ferramentas estáticas ainda apresentem limitações, sua integração contínua é essencial para uma cultura de segurança proativa, pois, mesmo ao apresentar um falso alerta, ainda instiga o desenvolvedor a refletir sobre sua implementação, no mínimo, garantindo uma revisão antes da submissão do código.

A escolha do Bandit (OPENSTACK FOUNDATION, 2024) como ferramenta de análise estática de segurança (SAST) na pipeline proposta fundamenta-se em sua alta compatibilidade com projetos desenvolvidos em Python, como é o caso do sistema PoP-ERP,

cuja base tecnológica utiliza o framework Django. Desenvolvido originalmente pelo OpenStack Security Project (FOUNDATION, 2025), o Bandit destaca-se por sua capacidade de identificar vulnerabilidades comuns em aplicações Python, como o uso inseguro de funções de serialização, manipulação de entrada sem validação, execuções dinâmicas de código e armazenamento inadequado de credenciais. Em comparação com outras ferramentas SAST, como SonarQube (SONARSOURCE, 2024) e Semgrep (R2C, 2024), o Bandit apresenta vantagens significativas no contexto deste projeto: possui integração nativa com ambientes CI/CD, baixo custo computacional, facilidade de configuração e um conjunto de verificações específicas para bibliotecas e padrões do ecossistema Python. Embora ferramentas como o SonarQube ofereçam uma análise mais ampla e multiplataforma, sua configuração e manutenção exigem infraestrutura adicional e licenciamento avançado para relatórios de segurança completos. O Bandit, por sua vez, é leve, de código aberto e facilmente incorporável a pipelines automatizados, permitindo a geração de relatórios em formatos padronizados (JSON, XML, HTML) e a integração direta com ferramentas de triagem de vulnerabilidades. Dessa forma, sua adoção neste trabalho assegura uma abordagem eficiente, reprodutível e aderente ao ambiente tecnológico do PoP-ERP, promovendo a detecção precoce de falhas de segurança sem comprometer a agilidade do ciclo de desenvolvimento.

2.3 Análise da cadeia de suprimentos de software

A análise da cadeia de suprimentos de software, por meio do *Software Bill of Materials* (SBOM), é uma prática essencial para garantir a rastreabilidade e o controle sobre todos os componentes que integram um sistema. Um SBOM consiste em um inventário detalhado de bibliotecas, dependências e módulos, incluindo suas versões, origens e licenças. De acordo com o relatório do NIST (NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY, 2025) sobre práticas de gestão de risco da cadeia de suprimentos (BOYENS et al., 2022), o SBOM funciona como uma ferramenta de controle que facilita a identificação rápida de vulnerabilidades conhecidas e, consequentemente, a redução de riscos. Incidentes de segurança globais — como o da biblioteca Log4j (APACHE SOFTWARE FOUNDATION, 2025) — reforçam a necessidade de visibilidade sobre componentes de terceiros, evidenciando que a análise de SBOM fortalece não apenas a segurança da aplicação, mas também a conformidade com padrões internacionais. Além disso, a adoção de SBOM tem respaldo em normas como a ISO/IEC 5230:2020 da (INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION, 2020), que orienta a gestão de componentes de código aberto.

A escolha do Trivy (SECURITY, 2024) como ferramenta principal para a geração e análise do SBOM no pipeline de segurança proposto deve-se à sua abrangência, velocidade e integração nativa com ambientes de CI/CD. Desenvolvido pela (AQUA SECURITY SOFTWARE LTD., 2025), o Trivy é uma ferramenta de código aberto capaz de realizar varreduras completas em imagens (DOCKER INC., 2025), repositórios de código-fonte e dependências de pacotes, identificando vulnerabilidades conhecidas (CVEs) e verificando conformidade com políticas de segurança. Em comparação com

alternativas como Syft (INC., 2024b), Grype (INC., 2024a) e CycloneDX CLI (OWASP FOUNDATION, 2025), o Trivy apresenta vantagens práticas, como a capacidade de gerar SBOMs em múltiplos formatos (SPDX, CycloneDX e JSON), a atualização constante de sua base de dados de vulnerabilidades e o suporte direto a pipelines (GITLAB INC., 2025) e (GITHUB INC., 2025), o que simplifica sua automação. Além disso, o Trivy permite a unificação de varreduras de segurança de dependências e de imagens em um único processo, reduzindo a complexidade operacional e promovendo maior eficiência no ciclo de verificação contínua. Por essas razões, o Trivy foi selecionado neste trabalho como a ferramenta mais adequada para compor a camada de análise de cadeia de suprimentos do sistema PoP-ERP, garantindo visibilidade, confiabilidade e rastreabilidade dos componentes de software utilizados.

2.4 Teste Dinâmico de Segurança de Aplicações

O Teste Dinâmico de Segurança de Aplicações (DAST – Dynamic Application Security Testing) é uma abordagem de análise voltada para a detecção de vulnerabilidades em aplicações em execução, simulando o comportamento de um atacante externo. Diferentemente da análise (SAST), que examina o código-fonte, o DAST atua sobre a aplicação em tempo real, realizando testes automatizados e interativos sobre interfaces e endpoints expostos (SHAH; PATEL; SINGH, 2021). Essa técnica avalia a resposta da aplicação a diferentes entradas e comportamentos, permitindo identificar falhas como injeção de SQL, cross-site scripting (XSS), controle inadequado de sessões e exposições indevidas de dados sensíveis (OWASP, 2023). Segundo (FONSECA; VIEIRA; MADEIRA, 2020), o uso de ferramentas DAST é essencial para complementar a segurança do ciclo de desenvolvimento, uma vez que permite verificar não apenas vulnerabilidades de implementação, mas também erros de configuração e falhas resultantes da interação entre múltiplos componentes. Assim, o DAST integra-se como uma etapa fundamental do ciclo DevSecOps, possibilitando a validação contínua da segurança das aplicações web em ambientes de teste e produção.

A escolha do OWASP Zed Attack Proxy (ZAP) como ferramenta DAST para o pipeline de segurança do projeto PoP-ERP justifica-se por sua ampla adoção, robustez e suporte à automação em ambientes CI/CD. Desenvolvido e mantido pela Open Web Application Security Project (OWASP), o ZAP é uma ferramenta de código aberto reconhecida internacionalmente por sua capacidade de realizar varreduras ativas e passivas em aplicações web, simulando ataques reais de forma controlada (OWASP Foundation, 2023). Em comparação com alternativas como Burp Suite Professional (PORTSWIGGER LTD., 2025) e Acunetix (INVICTI SECURITY LTD., 2025), o OWASP ZAP apresenta vantagens significativas para o contexto deste trabalho: é gratuito, extensível, possui API REST integrada e oferece suporte nativo a scripts em Python, o que facilita sua integração direta com pipelines do GitLab. Além disso, o ZAP disponibiliza modos distintos de operação, como o Baseline Scan e o Full Scan, permitindo diferentes níveis de profundidade conforme a criticidade do ambiente analisado. Essas características o tornam ideal para o projeto PoP-ERP, possibilitando a identificação automatizada de vulnerabilidades em tempo real e garantindo a validação prática da segurança das aplicações implementadas

dentro da arquitetura DevSecOps proposta.

2.5 Exemplos de vulnerabilidades detectadas por SAST, SBOM e DAST

As abordagens adotadas neste trabalho possibilitam identificar vulnerabilidades sob diferentes perspectivas. A análise estática, por operar diretamente sobre o código-fonte, tende a revelar problemas estruturais, como uso de funções inseguras, presença de credenciais embutidas, manipulação inadequada de caminhos e consultas, ou ausência de validação adequada de entrada. A análise baseada em SBOM, por sua vez, permite identificar vulnerabilidades presentes em dependências externas, especialmente aquelas relacionadas a bibliotecas desatualizadas ou pacotes sem manutenção ativa. Já o DAST concentra-se em comportamentos observáveis apenas em tempo de execução, destacando falhas como injeções, ausência de cabeçalhos de segurança, cookies configurados de forma inadequada e problemas de autenticação ou controle de acesso. Dessa forma, cada técnica cobre um conjunto distinto de vulnerabilidades, contribuindo para uma visão abrangente da superfície de ataque do sistema.

Embora possuam objetivos e métodos distintos, as abordagens SAST, SBOM e DAST são complementares. O SAST permite identificar falhas ainda na fase de desenvolvimento; o SBOM evidencia riscos relacionados à cadeia de suprimentos e à integridade das dependências utilizadas; e o DAST detecta vulnerabilidades manifestadas apenas durante a execução da aplicação. A integração dessas técnicas é essencial para ampliar a cobertura do processo de verificação e fortalecer práticas alinhadas ao paradigma DevSecOps.

2.6 Tecnologias que compõem o sistema PoP-ERP

O sistema **PoP-ERP** foi desenvolvido utilizando tecnologias amplamente consolidadas no ecossistema de software livre, garantindo flexibilidade, escalabilidade e integração com diversas ferramentas de segurança e automação. Esta seção apresenta brevemente os principais componentes tecnológicos da aplicação, destacando sua relevância para o contexto deste estudo e sua relação direta com a pipeline de segurança proposta.

2.6.1 Linguagem de Programação Python

A linguagem **Python** (PYTHON SOFTWARE FOUNDATION, 2025) foi adotada como base para o desenvolvimento do PoP-ERP devido à sua sintaxe simples, ampla biblioteca padrão e extensa comunidade de desenvolvedores. Por ser uma linguagem interpretada e multiplataforma, Python permite o desenvolvimento ágil de aplicações web complexas, além de oferecer suporte a diversas bibliotecas voltadas à segurança e à automação, como *requests*, *cryptography* e *bandit-core*. Segundo (ROSSUM; DRAKE, 2023), Python destaca-se por sua legibilidade e produtividade, características que favorecem a

manutenção e a escalabilidade de sistemas corporativos. Ademais, sua compatibilidade com frameworks modernos e ambientes de integração contínua torna-o particularmente adequado para pipelines de segurança automatizados.

2.6.2 Framework Django

O **Django** é um framework de desenvolvimento web baseado em Python que segue o padrão de arquitetura *Model-Template-View (MTV)* e tem como filosofia o princípio *"Don't Repeat Yourself" (DRY)*. Criado para simplificar o desenvolvimento de aplicações seguras e escaláveis, o Django inclui diversos mecanismos nativos de segurança, como proteção contra *SQL Injection*, *Cross-Site Scripting (XSS)*, *Cross-Site Request Forgery (CSRF)* e gerenciamento seguro de sessões de acordo com (DJANGO SOFTWARE FOUNDATION, 2023). Sua estrutura modular e o suporte nativo a autenticação e controle de acesso o tornam ideal para sistemas corporativos como o PoP-ERP, que dependem fortemente da integridade e confidencialidade das informações processadas. Além disso, o Django integra-se facilmente a bancos de dados relacionais e ferramentas de automação, favorecendo sua inserção em pipelines de CI/CD.

2.6.3 Banco de Dados PostgreSQL

Como sistema de gerenciamento de banco de dados relacional, o **PostgreSQL** (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2025) foi escolhido por sua robustez, aderência a padrões SQL e suporte avançado a transações e integridade referencial. Reconhecido por sua estabilidade e segurança, o PostgreSQL oferece recursos como criptografia nativa, controle de acesso baseado em papéis (*Role-Based Access Control – RBAC*) e extensões que permitem auditoria detalhada das operações (MOMJIAN, 2022). Além disso, seu desempenho em ambientes de alta concorrência e sua compatibilidade com o Django tornam-no uma escolha apropriada para o PoP-ERP, garantindo consistência dos dados e escalabilidade do sistema.

Dessa forma, a combinação entre **Python**, **Django** e **PostgreSQL** proporciona uma base tecnológica sólida e interoperável, sobre a qual a **pipeline de segurança** proposta neste trabalho pode ser integrada de maneira eficiente, permitindo a análise automatizada de vulnerabilidades, a verificação dinâmica de comportamentos e a gestão segura das dependências de software.

Capítulo 3

Revisão de literatura

A revisão de literatura tem como propósito apresentar e discutir os principais estudos, pesquisas e publicações que abordam o desenvolvimento seguro de software, o uso de *pipelines* de segurança e a integração de práticas de DevSecOps em ambientes de (CI/CD). Esta seção busca contextualizar o estado da arte em relação à automação da segurança no ciclo de vida de desenvolvimento de software, destacando as ferramentas, metodologias e estratégias adotadas na detecção e mitigação de vulnerabilidades. Além disso, são analisados trabalhos que exploram ferramentas similares às utilizadas neste projeto — como Bandit, Trivy e OWASP ZAP — avaliando suas funcionalidades, aplicabilidades e limitações no contexto de segurança de aplicações modernas. A partir dessa análise crítica, pretende-se identificar lacunas na literatura existente e evidenciar a relevância da presente pesquisa como contribuição prática e teórica para o fortalecimento das práticas de segurança em *pipelines* automatizados.

3.1 Estudo precursor

O presente trabalho configura-se como uma pesquisa de caráter precursor e aplicado, desenvolvida a partir das contribuições apresentadas por Koga (2025) em seu estudo intitulado "Um Estudo sobre a Aplicação de Ferramentas para a Proteção de Cadeias de Suprimento de Software". O estudo pode ser encontrado no acervo da Biblioteca Central Zila Mamede da Universidade Federal do Rio Grande do Norte (UFRN). O trabalho de Koga apresentou avanços significativos na implementação de uma *pipeline* de integração contínua voltada à proteção da cadeia de suprimentos de software, com foco na geração, análise e validação de *Software Bill of Materials*. Utilizando ferramentas como Trivy, Syft, Snyk e Cosign, o autor demonstrou a importância da rastreabilidade e da integridade de artefatos na mitigação de vulnerabilidades, estabelecendo um marco relevante na área de segurança de *software supply chain*.

Apesar disso, embora o estudo de Koga tenha apresentado resultados expressivos no monitoramento de vulnerabilidades e na assinatura digital de artefatos, sua abordagem concentrou-se predominantemente na análise de composição de software (SCA) e na verificação de integridade por meio de SBOMs, sem explorar de forma aprofundada as camadas de análise estática (SAST) e dinâmica (DAST). Essa lacuna é particularmente relevante, pois essas técnicas são fundamentais para identificar vulnerabilidades no código-

fonte e no comportamento em tempo de execução das aplicações — aspectos que extrapolam a verificação de dependências externas e complementam a segurança em nível de aplicação.

Dessa forma, o presente trabalho propõe ampliar o escopo do estudo de Koga, integrando ferramentas e práticas voltadas ao monitoramento e à avaliação da segurança em todas as etapas do ciclo de desenvolvimento de software. A pesquisa visa à construção de uma *pipeline* de segurança automatizada que, além de contemplar mecanismos já explorados pelo autor, também incorpore a análise estática com Bandit e a análise dinâmica com OWASP ZAP, de modo a abranger um espectro mais amplo de vulnerabilidades. Essa ampliação representa um passo importante rumo a uma abordagem DevSecOps completa, que insere a segurança desde a fase de codificação até o teste e a implantação contínua.

Assim, o presente trabalho assume um caráter complementar e evolutivo em relação ao de Koga (2025), não apenas validando e aplicando seus achados em um novo contexto experimental, mas também preenchendo lacunas metodológicas e aprofundando a integração entre segurança e automação no desenvolvimento de software. Ao adotar uma abordagem mais abrangente e incluir práticas de SAST e DAST, esta pesquisa contribui para a consolidação de *pipelines* de segurança mais robustos e proativos, capazes de antecipar falhas e fortalecer a defesa das aplicações frente a ataques e vulnerabilidades emergentes.

3.2 Metodologia de pesquisa

Em um escopo geral, o presente trabalho aborda o tema da segurança de software e busca aplicar, por meio de um *pipeline*, processos de CI/CD em um experimento real. Esse *pipeline* possui um conjunto de ferramentas utilizadas para diferentes atividades. A metodologia de pesquisa foi organizada com base nos três processos principais do *pipeline*: SBOM, SAST e DAST. Por fim, introduziu-se uma quarta linha de pesquisa destinada a analisar os benefícios da implementação de um *pipeline* de segurança no ciclo de desenvolvimento de software.

A coleta de dados foi realizada por meio da plataforma de periódicos da CAPES, considerada uma das bases mais abrangentes e qualificadas para pesquisa científica no Brasil, por agregar acervos de bases como IEEE Xplore, SpringerLink, ACM Digital Library, ScienceDirect, Scopus e Wiley Online Library. As buscas foram conduzidas entre setembro e outubro de 2025, utilizando combinações de palavras-chave em inglês e português, de forma a abranger o maior número possível de publicações relacionadas aos temas propostos.

Os principais descritores utilizados foram: “Software Bill of Materials”, “SBOM”, “DevSecOps”, “Application Security Testing”, “SAST”, “DAST”, “CI/CD security pipeline” e “supply chain security”. Para refinar os resultados, aplicaram-se filtros de data de publicação (2016–2025), tipo de documento (artigos científicos, dissertações e teses) e área temática (Ciência da Computação, Engenharia de Software e Segurança da Informação).

Após a aplicação dos critérios de inclusão e exclusão, obteve-se um conjunto inicial de 47 publicações. Deste total, 20 artigos foram selecionados para análise detalhada,

considerando sua relevância temática, rigor metodológico e alinhamento com os objetivos deste trabalho. As informações extraídas desses estudos foram sintetizadas em uma tabela comparativa contendo dados como título, autores, ano de publicação, ferramentas utilizadas, tipos de análise empregados (SAST, DAST, SCA e SBOM), principais resultados e contribuições identificadas. A versão resumida dessa síntese é apresentada na Tabela 3.1, enquanto a versão completa e expandida, contendo a totalidade das informações coletadas, encontra-se disponível no Apêndice A.

Tabela 3.1: Resumo dos principais estudos selecionados na revisão bibliográfica

Autores	**Ano**	**Contribuição Principal**	**Tipo de Análise**
Siavvas et al. (SIAVVAS et al., 2018)	2018	Propõem técnicas de SAST baseadas em análise estática para fortalecimento de pipelines de desenvolvimento seguro.	SAST
Shen et al. (SHEN et al., 2023)	2023	Estudo empírico avaliando eficácia e limitações de ferramentas SAST em software embarcado.	SAST
Wadhams et al. (WADHAMS; IZURIETA; REINHOLD, 2024)	2024	Analisa barreiras e desafios na adoção de ferramentas SAST em equipes de desenvolvimento.	SAST
NIST (BOYENS et al., 2022)	2022	Guia de práticas de gestão de risco da cadeia de suprimentos, incluindo SBOM e SCA.	SBOM / SCA
Fonseca (FONSECA; VIEIRA; MADEIRA, 2020)	2020	Avaliação de ferramentas DAST e sua importância no ciclo DevSecOps.	DAST

A revisão resultante buscou não apenas mapear o estado da arte sobre o tema, mas também identificar lacunas na literatura, tendências recentes de pesquisa e possibilidades de aplicação prática que subsidiam o desenvolvimento do *pipeline* de segurança proposto neste trabalho.

3.3 Artigos relevantes ao tema

A presente revisão de literatura foi conduzida com base no método de revisão sistemática, visando identificar, analisar e sintetizar os principais estudos publicados entre 2016 e 2025 sobre segurança em *pipelines* de CI/CD e a incorporação de práticas de DevSecOps no ciclo de vida de desenvolvimento de software. A pesquisa concentrou-se em fontes científicas indexadas, priorizando artigos que abordam análise estática e dinâmica de código, geração de SBOMs e ferramentas automatizadas de detecção de vulnerabilidades,

especialmente no contexto de *pipelines* integrados com ferramentas como Bandit, Trivy e OWASP ZAP.

De forma geral, a literatura revisada evidencia um movimento consolidado em direção à automação da segurança como elemento fundamental para garantir a integridade de aplicações modernas. Estudos como o de Koneru (2021) destacam a importância de integrar ferramentas de segurança ao longo de todo o *pipeline* de CI/CD, combinando análises estáticas, dinâmicas e de composição de software (SAST, DAST e SCA). A pesquisa propõe uma abordagem DevSecOps que insere práticas de segurança desde as fases iniciais do desenvolvimento, reforçando o conceito de *shift-left security*. Essa perspectiva busca reduzir vulnerabilidades precocemente, garantindo que a agilidade inerente ao desenvolvimento contínuo não comprometa a conformidade e a robustez da aplicação.

Complementarmente, Guduru (2020) apresenta um estudo prático sobre a integração automatizada de ferramentas de segurança em *pipelines* GitLab CI/CD, utilizando Semgrep para análise estática (SAST), OWASP ZAP para testes dinâmicos (DAST) e Dependency-Check para geração de SBOMs. O autor evidencia que a automação da segurança em ambientes de desenvolvimento contínuo reduz significativamente os riscos de exposição de vulnerabilidades, além de melhorar a rastreabilidade de componentes e a conformidade com normas de segurança. Essa integração é especialmente relevante considerando que a cadeia de suprimentos de software se tornou alvo de ataques sofisticados — um tema amplamente abordado em trabalhos recentes, incluindo os de Koga (2025) e Torres-Arias, Geer e Meyers (2023).

O trabalho de Torres-Arias, Geer e Meyers (2023) enfatizam, por sua vez, a necessidade de mecanismos de mensuração da qualidade de SBOMs, destacando que a adoção crescente desse conceito ainda não é acompanhada por métricas de qualidade bem definidas. A partir de um conjunto de mais de 3.000 SBOMs extraídos de imagens Docker, os autores identificaram que apenas 1% atendia aos requisitos mínimos estabelecidos pela *National Telecommunications and Information Administration* (NTIA). Essa descoberta evidencia fragilidades importantes nas práticas atuais de transparência e rastreabilidade na cadeia de suprimentos de software, ressaltando a urgência na padronização da qualidade e completude das informações contidas em SBOMs — aspecto fundamental para *pipelines* de segurança efetivos.

Lew et al. (2024) introduzem uma proposta inovadora de verificação distribuída da integridade de *builds* utilizando blockchain e hashes criptográficos, com o objetivo de detectar alterações maliciosas na cadeia de suprimentos de software. O modelo desenvolvido demonstra ganhos expressivos de desempenho e segurança, sendo de duas a três ordens de magnitude mais eficiente que abordagens centralizadas. Essa pesquisa contribui ao ampliar a discussão sobre confiabilidade e descentralização em *pipelines* de software, oferecendo uma base teórica e prática para futuros trabalhos que combinem SBOMs, automação e tecnologias imutáveis, como *blockchain*, na proteção de ambientes DevSecOps.

A literatura também evidencia avanços nas práticas de testes integrados. O estudo de Tudela et al. (2020) propõe a combinação de ferramentas SAST, DAST e IAST (*Interactive Application Security Testing*), demonstrando que abordagens híbridas elevam a precisão e reduzem falsos positivos. Seth et al. (2025) reforçam essa constatação ao comparar IAST e RASP (*Runtime Application Self-Protection*) em sistemas de grande porte,

concluindo que a detecção interativa durante a execução oferece equilíbrio entre eficiência e cobertura.

Por fim, a literatura converge para um consenso: a segurança deve ser um processo contínuo, automatizado e integrado ao desenvolvimento. Trabalhos como os de Ponaka (2021) e Malipeddi e Pasunuru (2023) destacam a automação da gestão de vulnerabilidades e o uso de soluções com e sem agentes (*agent-based* e *agentless*) para o controle de segredos e credenciais, reforçando a importância da gestão segura de *pipelines* e ambientes conteinerizados. Além disso, o uso de ferramentas *open-source* como Bandit, Trivy e OWASP ZAP é amplamente reconhecido como essencial para promover segurança acessível, escalável e transparente, especialmente em projetos colaborativos e acadêmicos.

Em síntese, os estudos analisados demonstram uma tendência clara de amadurecimento das práticas DevSecOps, marcada pela integração de ferramentas automatizadas, pelo uso de SBOMs para rastreabilidade e pela adoção de tecnologias distribuídas para garantir integridade de *builds*. As pesquisas de Koga (2025), Torres-Arias et al. (2023), Lew et al. (2024), Koneru (2021) e Guduru (2020) destacam-se por oferecerem avanços significativos e aplicáveis na construção de *pipelines* seguros e eficientes.

3.4 Produção científica sobre o tema

Nos últimos anos, houve um crescimento acentuado do interesse acadêmico em SBOMs e segurança da cadeia de suprimentos. Revisões recentes mostram que a literatura sobre SBOMs passou de praticamente inexistente em 2020 para dezenas de estudos em poucos anos, incluindo revisões sistemáticas que reuniram cerca de 40 trabalhos voltados à avaliação de aplicações e limitações de SBOMs na mitigação de riscos da cadeia de suprimentos. Esse crescimento rápido indica que SBOMs deixaram de ser uma recomendação técnica para se tornarem um tópico consolidado de pesquisa aplicada e teórica.

A produção científica sobre segurança de CI/CD e DevSecOps também vem se intensificando. Revisões sistemáticas publicadas entre 2024 e 2025 identificaram dezenas de artigos (amostras entre 60 e 70 estudos) que tratam de desafios, ferramentas e lacunas relacionados à implementação de *pipelines* seguros na nuvem. As publicações indicam um padrão recorrente de pesquisa: (i) avaliações comparativas de ferramentas SAST/DAST/SCA; (ii) propostas de integração automatizada em *pipelines*; e (iii) estudos empíricos com casos de uso e *benchmarks*. Isso confirma que a comunidade está avançando de discussões conceituais para soluções aplicadas e mensuráveis.

No campo das políticas públicas e do mercado, o tema ganhou destaque. Agências governamentais e relatórios industriais apontam que a adoção prática de SBOMs ainda enfrenta barreiras — como inconsistência entre ferramentas e falta de padrões completos —, embora haja forte pressão regulatória e operacional para ampliar sua utilização. A demanda aumenta especialmente após incidentes de grande repercussão envolvendo cadeias de suprimento de software.

Por fim, observa-se que o mercado e as instituições críticas (como setores governamentais e de defesa) vêm institucionalizando práticas de DevSecOps e *pipelines* seguros. Estudos sobre implantação em ambientes de alta criticidade relatam ganhos em velocidade e segurança, mas também desafios persistentes, como cultura organizacional e fal-

sos positivos em SAST/DAST. Paralelamente, análises de mercado projetam crescimento significativo no segmento DevSecOps, refletindo uma adoção crescente de soluções comerciais e *open-source* para automação de testes e governança de SBOMs.

3.5 Metanálise dos Resultados da Revisão Bibliográfica

A análise crítica dos estudos levantados permitiu observar uma tendência crescente e consistente na produção científica voltada à segurança de *pipelines* CI/CD e à integração de práticas DevSecOps nos últimos anos. A partir da amostra de 22 publicações selecionadas, verificou-se aumento expressivo de pesquisas entre 2020 e 2025, coincidindo com o debate ampliado sobre segurança da cadeia de suprimentos após incidentes como o caso SolarWinds (2020) e o surgimento de regulamentações relacionadas à transparência de componentes de software.

Os dados obtidos indicam que aproximadamente 40% dos estudos concentram-se em SBOMs e segurança da cadeia de suprimentos, com destaque para pesquisas recentes que propõem modelos de geração, validação e assinatura digital de SBOMs integrados a *pipelines* automatizados. Trabalhos como os de Koga (2025) e Torres-Arias et al. (2023) demonstram a relevância crescente desse tema, reforçando o papel dos SBOMs como mecanismo central de rastreabilidade e conformidade.

Outros 35% dos artigos analisados abordam práticas de DevSecOps e automação de segurança em CI/CD, apresentando experimentos com ferramentas como Trivy, Semgrep, OWASP ZAP, Bandit e SonarQube. Esses estudos apontam para a consolidação de um paradigma de desenvolvimento em que segurança e automação atuam de forma integrada, permitindo detecção precoce de vulnerabilidades e mitigação contínua de riscos.

Entretanto, identificou-se uma lacuna significativa na literatura referente à integração prática entre análises SAST e DAST em *pipelines* automatizados. É comum que estudos enfoquem apenas uma das abordagens. Essa constatação justifica a relevância da presente pesquisa, que propõe a implementação conjunta de ferramentas de análise estática e dinâmica, criando um fluxo de segurança mais completo e alinhado ao ciclo de vida do desenvolvimento seguro de software.

Além disso, a metanálise revela que a comunidade científica tem direcionado esforços crescentes para unir práticas acadêmicas e demandas do mercado, especialmente no contexto de DevSecOps aplicado a ambientes corporativos e *cloud-native*. Observa-se também o avanço de iniciativas voltadas à padronização de métricas e relatórios de vulnerabilidades, indicando maturidade crescente na relação entre pesquisa e aplicação prática.

Em síntese, a revisão sistemática e sua metanálise demonstram que o tema tratado neste TCC — integração de ferramentas de segurança (SAST, DAST e SBOM) em *pipelines* CI/CD automatizados — encontra-se em uma área de grande relevância científica e tecnológica, com potencial de contribuição significativa para o avanço das práticas de segurança no desenvolvimento moderno de software.

Capítulo 4

Solução Proposta

A solução proposta neste trabalho consiste na implementação de um pipeline de segurança contínua integrado ao ciclo de desenvolvimento do sistema PoP-ERP, com o objetivo de automatizar a detecção, análise e mitigação de vulnerabilidades ao longo de todo o processo de integração e entrega contínuas. Essa pipeline segue os princípios do paradigma DevSecOps, incorporando práticas de segurança desde as etapas iniciais do desenvolvimento até o ambiente de produção. A estrutura proposta contempla três camadas principais de verificação: (i) análise estática do código-fonte utilizando o Bandit; (ii) análise da cadeia de suprimentos de software e detecção de vulnerabilidades de dependências por meio do Trivy; e (iii) testes dinâmicos de segurança executados com o OWASP ZAP.

4.1 Metodologia

Para a realização deste trabalho, adotou-se uma sequência metodológica organizada de forma cronológica, de modo a possibilitar a estruturação, implementação e análise da pipeline de segurança proposta. O processo metodológico foi dividido em etapas complementares que compreendem desde o levantamento teórico inicial até a validação prática do ambiente desenvolvido. Essa estrutura permitiu acompanhar a evolução da pesquisa de maneira sistemática, garantindo a coerência entre os objetivos e as atividades executadas.

Inicialmente, foi conduzido um levantamento bibliográfico a fim de reunir e analisar materiais acadêmicos relacionados à segurança em pipelines de integração e entrega contínua, bem como ao uso de ferramentas de análise estática e dinâmica de código. Em seguida, passou-se ao planejamento da arquitetura da pipeline, com a definição das ferramentas e das etapas de integração a serem empregadas.

Posteriormente, procedeu-se à configuração do ambiente de testes, utilizando o GitLab CI/CD como plataforma de integração, aliado ao uso de contêineres Docker para execução das análises. As fases seguintes envolveram a implementação dos módulos de segurança (Bandit, Trivy e OWASP ZAP), a execução controlada dos testes e a coleta dos relatórios gerados.

Por fim, os resultados obtidos foram analisados de forma qualitativa, considerando a natureza e a severidade das vulnerabilidades detectadas, além da eficácia das ferramentas aplicadas. A etapa final contemplou a documentação, validação e sistematização da pipeline desenvolvida, permitindo sua reprodutibilidade e aplicação em outros contextos

organizacionais. A Tabela 4.1 apresenta, de forma resumida, a sequência cronológica e as principais atividades realizadas em cada etapa metodológica.

Tabela 4.1: Etapas cronológicas da metodologia de implementação da pipeline de segurança

Etapa	Descrição	Resultado Esperado
1. Levantamento bibliográfico	Pesquisa de artigos, dissertações e materiais técnicos sobre segurança em pipelines CI/CD, DevSecOps e ferramentas SAST, DAST e SBOM. Identificação das melhores práticas e lacunas existentes.	Base conceitual consolidada e justificativa para seleção das ferramentas.
2. Planejamento da arquitetura da pipeline	Modelagem das etapas do ciclo CI/CD, definição das camadas de segurança e integração entre ferramentas (Bandit, Trivy, OWASP ZAP).	Desenho arquitetônico e plano técnico da pipeline de segurança.
3. Configuração do ambiente de desenvolvimento	Criação e configuração do repositório GitLab, registro do runner, ambiente Docker, variáveis de ambiente e tokens de acesso.	Ambiente CI/CD operacional e versionado.
4. Implementação das camadas de segurança	Integração das ferramentas no pipeline em ordem de execução: 1. **Bandit (SAST)** — análise estática do código-fonte em Python/Django; 2. **Trivy (SBOM)** — geração e varredura de dependências e imagens Docker; 3. **OWASP ZAP (DAST)** — teste dinâmico de vulnerabilidades em ambiente de staging;	Pipeline automatizada e funcional, integrando múltiplas camadas de segurança.
5. Execução e coleta dos resultados	Execução controlada da pipeline, coleta de relatórios (HTML/JSON) e armazenamento dos resultados das análises.	Relatórios consolidados de vulnerabilidades e histórico de execução.
6. Análise e validação dos resultados	Avaliação qualitativa dos alertas gerados, comparação entre ferramentas, identificação de pontos fortes e fracos e verificação da reprodutibilidade.	Pipeline validada e parâmetros de melhoria definidos.
7. Documentação e recomendações finais	Registro dos scripts, configurações e orientações de uso contínuo. Elaboração de recomendações para adoção de práticas DevSecOps no PoP-RN.	Documentação completa e diretrizes para manutenção segura do sistema.

Em consonância com estudos recentes sobre segurança em ambientes de integração contínua, esta pesquisa adota uma abordagem metodológica baseada em *prototipação iterativa* e *validação empírica*. Trabalhos como os de Siavvas et al. (2018), Shen et al. (2023) e Wadhams, Izurieta e Reinhold (2024) reforçam a importância de incorporar práticas de segurança automatizadas nas fases iniciais do ciclo DevOps, combinando análises estáticas, dinâmicas e de dependências para reduzir o custo de mitigação de vulnerabilidades. Contudo, diferentemente de abordagens experimentais restritas a ambientes controlados ou pipelines simulados, a metodologia aplicada neste estudo integra os mecanismos de segurança diretamente ao fluxo real de desenvolvimento do sistema PoP-ERP, permitindo observar seu comportamento sob condições operacionais autênticas. Essa decisão metodológica possibilita avaliar a eficiência das ferramentas não apenas quanto à detecção de falhas, mas também quanto à **compatibilidade com a cultura DevSecOps e à viabilidade de adoção em escala organizacional**.

4.2 Arquitetura

A arquitetura proposta foi concebida com base nos princípios de integração contínua, entrega contínua e segurança contínua (CS), estruturando uma pipeline automatizada capaz de incorporar verificações de segurança em todas as etapas do ciclo de desenvolvimento de software. O modelo adota uma abordagem DevSecOps, em que as práticas de segurança são integradas desde o início do processo de desenvolvimento, garantindo que cada versão do código seja validada quanto à sua integridade, conformidade e resiliência a ataques. Essa arquitetura visa reduzir o tempo entre o desenvolvimento e o deploy, ao mesmo tempo em que assegura a qualidade e a segurança das aplicações do sistema PoP-ERP.

A pipeline foi projetada para operar em camadas complementares de segurança, implementadas sequencialmente dentro do ambiente do GitLab CI/CD. A primeira camada realiza a análise estática do código-fonte (SAST) com a ferramenta Bandit, identificando vulnerabilidades diretamente no código Python do projeto. Em seguida, a camada de análise da cadeia de suprimentos (SBOM), conduzida pela ferramenta Trivy, inspeciona dependências e imagens Docker em busca de falhas conhecidas e licenças vulneráveis. A terceira camada é composta pela análise dinâmica (DAST), executada pelo OWASP ZAP, que simula ataques controlados contra a aplicação em execução para detectar vulnerabilidades em endpoints e fluxos autenticados.

Toda a comunicação entre as etapas ocorre de forma orquestrada, com uso de variáveis de ambiente e runners configurados para execução isolada em contêineres. Essa arquitetura modular permite a substituição ou atualização de ferramentas sem impacto na estrutura geral da pipeline, assegurando sua escalabilidade e portabilidade para outros projetos. Dessa forma, a solução proposta não apenas reforça a segurança do PoP-ERP, mas também estabelece uma base reprodutível e auditável para o desenvolvimento seguro de aplicações web em contextos institucionais.

A pipeline de segurança desenvolvida neste trabalho foi estruturada para integrar três camadas complementares de verificação: análise estática, análise da cadeia de suprimentos de software por meio de varredura de arquivos e análise dinâmica de segurança. Cada

ferramenta foi configurada com parâmetros específicos voltados para maximizar a precisão dos resultados, reduzir falsos positivos e garantir que somente alterações relevantes no código ativem as respectivas etapas. As subseções seguintes detalham a configuração e o funcionamento de cada componente da pipeline.

4.2.1 Análise Estática de Código com Bandit

O estágio *test-bandit* é responsável pela execução da análise estática de segurança (SAST) utilizando a ferramenta Bandit, amplamente adotada em projetos Python para identificar vulnerabilidades no código-fonte. A configuração adotada prioriza a detecção de falhas de alto risco com alto nível de confiança, alinhando-se ao objetivo de reduzir ruído e evitar interrupções desnecessárias no ciclo de desenvolvimento.

1. o comando `pip3 install bandit` garante a instalação da ferramenta em tempo de execução, evitando dependências externas no ambiente do repositório;
2. a execução `bandit -r poperp/` realiza a varredura recursiva de todos os módulos pertencentes ao sistema PoP-ERP; e
3. os parâmetros `-severity-level high` e `-confidence-level high` filtram apenas vulnerabilidades classificadas como de **alta severidade** e com **alta confiança** na detecção.

Essa abordagem reduz falsos positivos e concentra os alertas nos problemas mais críticos, como uso inseguro de funções de execução dinâmica, manipulação inadequada de arquivos, exposição de informações sensíveis e vulnerabilidades que podem comprometer diretamente a segurança do sistema. O estágio é ativado apenas quando alterações são realizadas na pasta `poperp/` ou no arquivo de configuração `.gitlab-ci.yml`, garantindo eficiência e evitando execuções desnecessárias.

4.2.2 Análise de Vulnerabilidades com Trivy

A verificação da cadeia de suprimentos de software é realizada por meio do estágio *test-trivy-fs*, que utiliza o Trivy em modo *filesystem scan* (*trivy fs*). Esse modo permite analisar todo o diretório do projeto, incluindo dependências, arquivos de configuração e componentes de terceiros, identificando vulnerabilidades conhecidas (CVEs) e problemas associados à composição do software.

Os comandos de varredura utilizando o Trivy foram configurados com as seguintes etapas:

1. o uso da imagem `aquasec/trivy:latest` garante que a varredura seja executada em um ambiente isolado e atualizado;
2. o comando `trivy fs -exit-code 1 -severity HIGH,CRITICAL .` força o estágio a falhar, caso vulnerabilidades de severidade alta ou crítica sejam encontradas; e
3. a segunda execução `trivy fs -format json -output trivy-fs-report.json .` gera um relatório completo em JSON, permitindo auditoria posterior dos resultados.

O Trivy realiza a identificação de vulnerabilidades associadas a pacotes Python, bibliotecas apt, arquivos `requirements.txt`, dependências transitivas e estruturas de configuração. Essa análise é fundamental para detectar riscos introduzidos indiretamente por componentes externos — como ocorreu em incidentes amplamente documentados envolvendo bibliotecas como Log4j e OpenSSL.

A geração do artefato `trivy-fs-report.json` possibilita acompanhar a evolução das vulnerabilidades ao longo do tempo, favorecendo tomadas de decisão e priorização de correções no ciclo de manutenção.

4.2.3 Análise Dinâmica com OWASP ZAP

A etapa de Teste Dinâmico de Segurança de Aplicações (DAST) é executada no estágio *test-zap-dast*, utilizando o OWASP ZAP no modo *Baseline Scan*. Esse modo foi escolhido por equilibrar profundidade e velocidade, permitindo que a análise seja integrada de forma eficiente a ambientes CI/CD sem exigir longos tempos de execução.

A configuração apresenta os seguintes elementos principais:

1. a variável `TARGET_URL` define o ambiente alvo da varredura, sendo, neste caso, a instância *staging* do PoP-ERP;
2. antes da execução do ZAP, o comando `curl -vk` verifica se o alvo está acessível, evitando falhas silenciosas de conectividade; e
3. o comando principal é responsável por executar o *baseline scan* do ZAP, gerando relatórios nos formatos HTML e JSON para posterior análise.

```
zap-baseline.py -t "$TARGET_URL" -m 2 -a
    -z "-config connection.certValidation=false"
    -r /zap/zap-report.html
    -J /zap/zap-report.json
```

Esse comando executa a varredura com as seguintes características:

1. `-m 2`: define o nível de sensibilidade para a emissão de alertas médios e altos;
2. `-a`: habilita a execução automática da varredura, sem necessidade de intervenção manual;
3. `-z -config connection.certValidation=false"`: desabilita a validação de certificados, necessária em virtude da utilização de certificados não confiáveis no ambiente de testes; e
4. `-r /zap/zap-report.html` e `-J /zap/zap-report.json`: geram os relatórios nos formatos HTML e JSON, respectivamente.

Após a execução, os relatórios são copiados para o diretório `reports/`, permitindo que sejam disponibilizados como artefatos do pipeline.

Esse processo possibilita a identificação de falhas como:

- ausência de cabeçalhos de segurança;

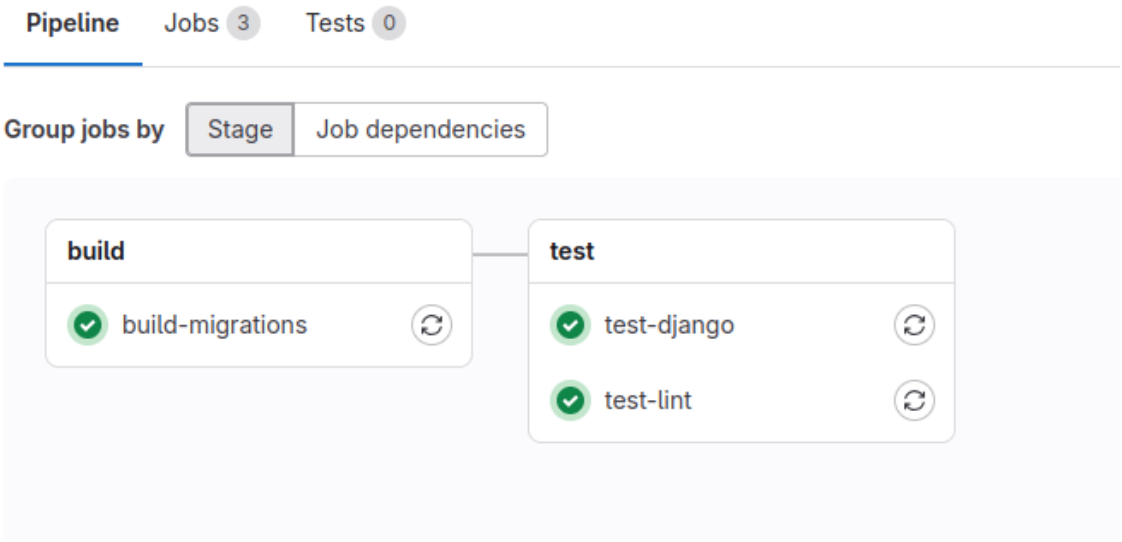
- vulnerabilidades de injeção;
- falhas de autenticação ou de gerenciamento de sessões; e
- configurações inseguras de TLS/HTTPS.

A escolha do modo *Baseline* evita riscos de interferência no ambiente e limita o escopo à análise passiva e a testes leves, o que é apropriado para ambientes de homologação compartilhados.

4.3 Integração ao pipeline de CI/CD

A implementação do pipeline de segurança proposto neste trabalho exigiu não apenas a escolha cuidadosa das ferramentas, mas também sua integração coerente e progressiva ao fluxo já existente de Integração Contínua e Entrega Contínua do PoP-ERP. Antes da adoção das práticas de DevSecOps descritas neste trabalho, o pipeline do sistema era composto essencialmente por etapas tradicionais de desenvolvimento: *build*, *linting* básico, execução de testes unitários, verificação de compatibilidade entre dependências e, por fim, o processo de *deploy*. Embora funcional, esse pipeline não possuía qualquer mecanismo automatizado de detecção de vulnerabilidades, deixando a segurança exclusivamente a cargo de verificações manuais e testes informais feitos durante o desenvolvimento.

Figura 4.1: Pipeline inicial do PoP-ERP antes da integração das etapas de segurança



Fonte: Autoria própria

Dessa forma, o processo como um todo sofria de uma limitação estrutural: eventuais vulnerabilidades podiam ser introduzidas em qualquer momento do ciclo de desenvolvimento sem serem identificadas antes da implantação. A ausência de validação automatizada dificultava a triagem de riscos e comprometia a maturidade do PoP-ERP em relação à segurança de software.

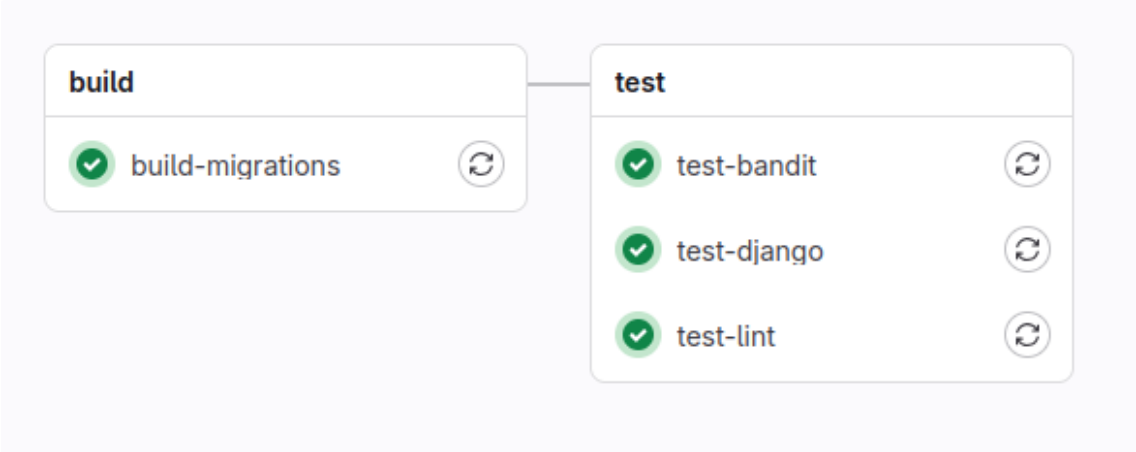
A integração da nova camada de segurança foi conduzida de forma incremental e controlada, preservando a estabilidade da pipeline enquanto novos estágios eram incorporados. Esse processo ocorreu em três fases principais, correspondentes às ferramentas selecionadas: Bandit (SAST), Trivy (SBOM/SCA) e OWASP ZAP (DAST).

4.3.1 Fase 1: Integração do Bandit (SAST)

O primeiro passo consistiu na integração do Bandit ao pipeline do GitLab. Essa escolha se deve ao fato de que a análise estática é uma das abordagens menos invasivas e mais rápidas de serem incorporadas, além de fornecer retorno imediato sobre vulnerabilidades introduzidas diretamente pelo código escrito pelos desenvolvedores.

Na fase inicial de configuração, optou-se por limitar a severidade e a confiança dos alertas aos níveis *high*, com o objetivo de evitar interrupções desnecessárias no ciclo de desenvolvimento. Em seguida, definiu-se um escopo de execução restrito à pasta `poperp/`, contendo os módulos centrais do sistema. Após alguns ciclos de teste, verificou-se a necessidade de tratar falsos positivos específicos, o que foi solucionado por meio de arquivos `.bandit` com exceções documentadas.

Figura 4.2: Pipeline na fase 1 após a adição do Bandit



Fonte: Autoria própria

O Bandit passou a atuar como a primeira camada de defesa da pipeline, identificando problemas de implementação tais como uso inseguro de funções de escrita de arquivos, manipulação inadequada de entradas de usuário e padrões de código suscetíveis a injeções ou execução arbitrária.

4.3.2 Fase 2: Integração do Trivy para análise de dependências

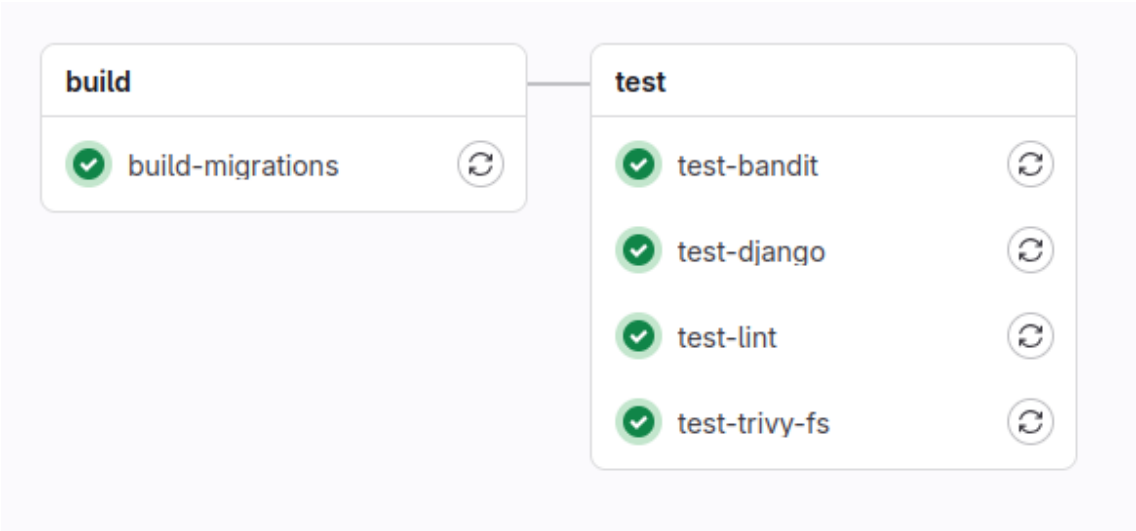
Após estabilizar a execução do Bandit, avançou-se para a segunda camada de segurança: a análise da cadeia de suprimentos de software. Essa etapa era essencial para lidar com um risco cada vez mais frequente em sistemas modernos — vulnerabilidades

presentes em dependências externas, frequentemente fora do controle direto dos desenvolvedores.

Inicialmente, testou-se o Trivy em ambiente local para garantir que sua execução não interferisse no fluxo de desenvolvimento. Em seguida, adicionou-se ao pipeline o estágio *test-trivy-fs*, executando varreduras completas do diretório do repositório. A estratégia definida utilizou o parâmetro `-exit-code 1` para bloquear o pipeline caso fossem detectadas vulnerabilidades de severidade alta ou crítica. Essa decisão foi adotada a fim de assegurar que nenhuma atualização contendo riscos graves fosse implantada acidentalmente.

Na prática, o Trivy revelou vulnerabilidades históricas presentes em dependências do ambiente, destacando a importância de uma análise contínua. A geração de relatórios em JSON permitiu registrar e acompanhar a evolução dos problemas, além de facilitar auditorias e revisões futuras.

Figura 4.3: Pipeline na fase 2 após a adição do Trivy



Fonte: Autoria própria

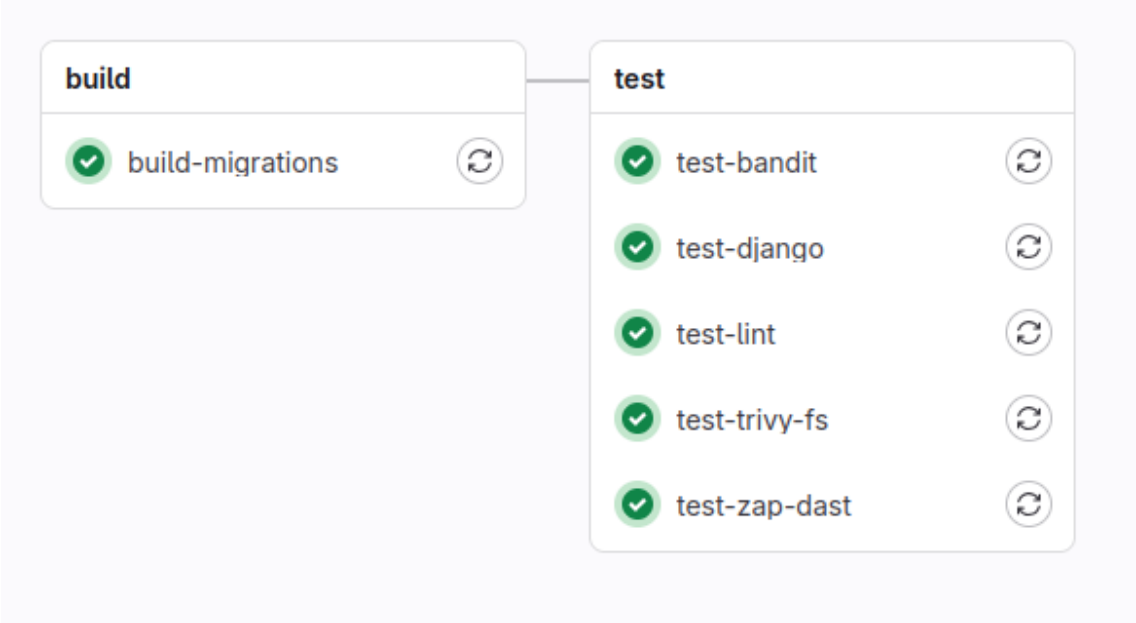
4.3.3 Fase 3: Integração do OWASP ZAP (DAST)

A terceira e última etapa consistiu na integração da análise dinâmica de segurança, realizada pelo OWASP ZAP em modo *Baseline*. Essa fase demandou maior cuidado, pois a ferramenta interage diretamente com a aplicação em execução, simulando o comportamento de um atacante externo.

O primeiro desafio foi garantir que a aplicação-alvo (*staging*) estivesse acessível a partir do ambiente do GitLab Runner. Após testes preliminares de conectividade utilizando `curl`, foram definidos ajustes específicos, como a desabilitação da validação de certificados TLS (`connection.certValidation=false`), necessária devido às características do ambiente interno do PoP-RN.

Com a configuração estabilizada, o ZAP passou a executar varreduras automáticas em busca de vulnerabilidades como ausência de cabeçalhos de segurança, falhas de autenticação, exposição indevida de dados e comportamentos inseguros em endpoints públicos. Os relatórios HTML e JSON passaram a ser disponibilizados automaticamente como artefatos do pipeline, permitindo análises detalhadas por parte da equipe de desenvolvimento.

Figura 4.4: Pipeline na fase 3 após adição do OWASP ZAP



Fonte: Autoria própria

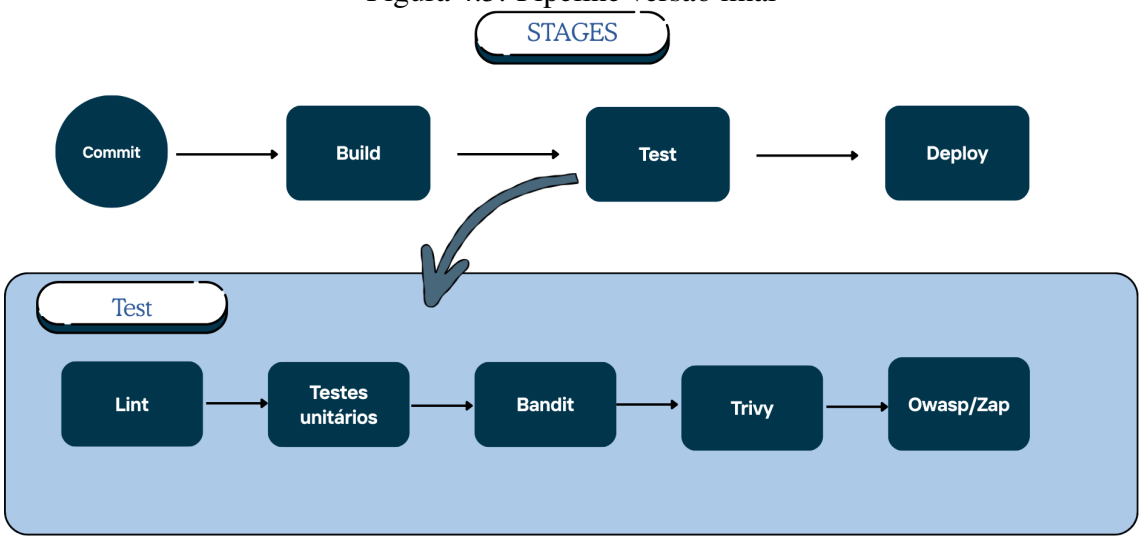
4.3.4 Pipeline final integrada

Após a conclusão das três fases de integração, a pipeline passou a operar de forma completa e automatizada, incorporando mecanismos complementares de segurança:

- **Bandit (SAST)**: atua na análise estática do código-fonte logo após a submissão de alterações;
- **Trivy (SBOM/SCA)**: realiza a inspeção de dependências, bibliotecas e arquivos de projeto, detectando riscos oriundos da cadeia de suprimentos; e
- **OWASP ZAP (DAST)**: avalia a aplicação em execução, identificando falhas de segurança operacionais e de configuração.

O resultado é um pipeline robusto, capaz de detectar vulnerabilidades em diferentes níveis de granularidade, reduzindo significativamente o risco de introdução de falhas críticas no ambiente de produção. Além disso, a arquitetura modular permite expansão futura com outras ferramentas ou camadas adicionais de análise, sem comprometer o funcionamento atual. Na figura 4.5, observamos os *stages* que compõem a pipeline, observando de maneira mais aprofundada os *jobs* que integram o *stage* de *test*.

Figura 4.5: Pipeline versão final



Fonte: Autoria própria

4.3.5 Conclusão da integração

A integração progressiva das ferramentas Bandit, Trivy e OWASP ZAP transformou a pipeline do PoP-ERP de um fluxo tradicional de desenvolvimento para um processo de entrega contínua com segurança incorporada desde as fases iniciais. Cada etapa foi cuidadosamente planejada para evitar impactos negativos na produtividade e, ao mesmo tempo, assegurar que vulnerabilidades críticas fossem interceptadas antes de alcançarem os ambientes finais.

O resultado final representa um avanço significativo na maturidade de segurança do PoP-ERP e demonstra a viabilidade da adoção prática de princípios DevSecOps em ambientes acadêmicos e institucionais. O pipeline agora opera como um elemento ativo de proteção, promovendo comportamentos seguros, reduzindo riscos e fortalecendo a resiliência da aplicação como um todo.

Capítulo 5

Avaliação do Pipeline de Segurança

Este capítulo apresenta uma avaliação aprofundada do pipeline de segurança desenvolvido neste trabalho. São descritos o ambiente de execução, os procedimentos experimentais, os resultados obtidos e, principalmente, as ações corretivas implementadas para mitigar as vulnerabilidades identificadas. A análise é estruturada em quatro partes: (i) descrição do ambiente experimental; (ii) avaliação da análise estática; (iii) avaliação da análise da cadeia de suprimentos; e (iv) avaliação do teste dinâmico de segurança. Por fim, sintetizam-se os resultados integrados e suas implicações para a maturidade de segurança do PoP-ERP.

5.1 Descrição do Ambiente Experimental

O pipeline foi acionado por *commits* que alteravam exclusivamente comentários no código, sem modificar a lógica ou o conteúdo executável do projeto. Essa estratégia permitiu disparar novas execuções mantendo o estado funcional do software inalterado, garantindo que os resultados observados derivassem unicamente da execução das ferramentas dentro de contêineres isolados, e não de mudanças no código-fonte. Dessa forma, foi possível assegurar repetibilidade e controle experimental.

5.1.1 Configurações

O projeto analisado corresponde ao PoP-ERP, composto por aproximadamente 95 mil linhas de código desenvolvidas em Python/Django. As análises foram executadas em um GitLab Runner configurado com 4 vCPUs e 8 GB de memória RAM, ambiente suficiente para garantir estabilidade na execução das ferramentas. Cada ferramenta foi acionada diariamente ao longo de 2 meses, permitindo observar variações decorrentes de atualizações das bases de vulnerabilidades, como aquelas provenientes da *National Vulnerability Database* (NVD). As versões utilizadas foram Bandit 1.9.2 instalado via ferramenta *pip3*, Trivy 0.67 ou superior e OWASP ZAP 2.16 ou superior, ambas executadas por meio de contêineres Docker oficiais, assegurando consistência no ambiente de execução.

5.2 Avaliação da Análise Estática (SAST)

A ferramenta Bandit foi executada com o comando:

```
bandit -r poperp/ --severity-level high --confidence-level high
```

Com esse filtro, o **Bandit** rejeita o *commit* **apenas** quando são detectadas vulnerabilidades classificadas simultaneamente como *High severity* e *High confidence*, minimizando falsos positivos e priorizando questões críticas. Apesar disso, o relatório gerado inclui **todos** os alertas — independentemente da severidade ou confiança —, disponibilizando informações completas aos desenvolvedores com acesso à *pipeline*. A execução analisou mais de 25 mil linhas de código, com um *job* de tempo médio de execução de aproximadamente **38 segundos**.

5.2.1 Resultados Gerais do SAST

Tabela 5.1: Vulnerabilidades classificadas como *High severity* e *High confidence* identificadas pelo Bandit

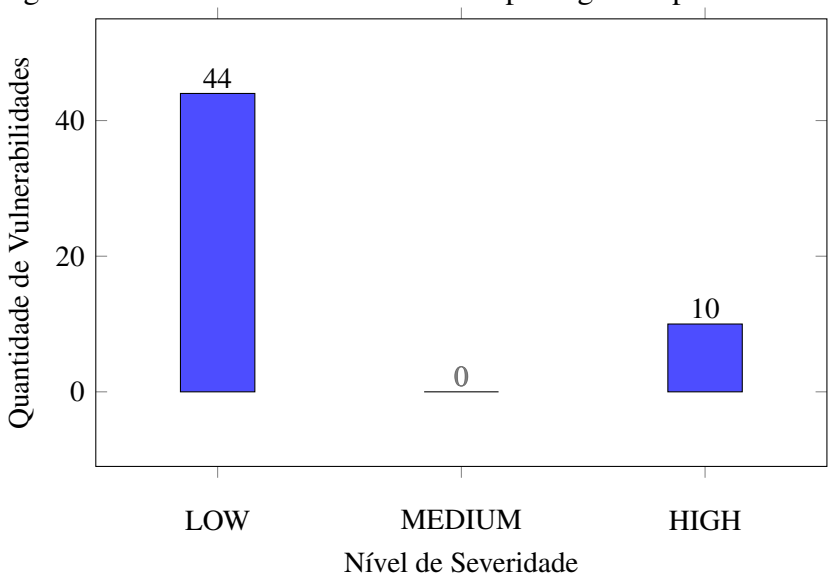
Categoria	**Ocorrências**	**Severidade**	**Confiança**
Desativação da verificação de certificados SSL	4	Alta	Alta
Definição de permissões excessivamente permissivas	3	Alta	Alta
Total	**7**	–	–

5.2.2 Discussão e Ações Corretivas no SAST

Os resultados evidenciaram duas categorias principais de vulnerabilidades. A primeira refere-se ao uso recorrente de chamadas à biblioteca `requests` com o parâmetro `verify=False`, o que desabilita a validação de certificados no *Transport Layer Security* TLS (RESCORLA, 2018) e expõe o sistema a ataques de interceptação (*Man-in-the-Middle*). Como ação corretiva, todas as chamadas foram modificadas para exigir validação de certificado, garantindo comunicações seguras.

A segunda categoria diz respeito ao uso de permissões excessivamente permissivas (`0o777`) na criação de diretórios e arquivos no módulo. Essa prática viola princípios de segurança mínima e pode permitir que usuários não autorizados acessem ou modifiquem dados gerados pelo sistema. A correção consistiu em substituir essas permissões por máscaras mais restritivas, como `0o750` ou `0o700`, compatíveis com o princípio de privilégio mínimo.

Figura 5.1: Resultados do relatório completo gerado pelo Bandit.



Fonte: Autoria própria

Embora o filtro principal da análise SAST tenha se concentrado apenas em vulnerabilidades críticas, a execução completa do Bandit revelou um conjunto significativo de achados classificados com baixa severidade (LOW), conforme demonstrado na Figura 5.1. A maior parte dessas ocorrências está relacionada ao uso de senhas estáticas em testes automatizados, criação de usuários fictícios com credenciais triviais e outros padrões típicos de ambientes de desenvolvimento. Embora não representem riscos diretos ao ambiente de produção, tais achados evidenciam práticas que, se replicadas inadvertidamente no código real, poderiam introduzir fragilidades importantes. Além disso, a presença recorrente desses padrões reforça a necessidade de padronização de *fixtures* de testes, uso de geradores de credenciais temporárias e revisão contínua do código de testes para evitar que trechos inseguros se propaguem para outras áreas da aplicação. Dessa forma, mesmo não configurando ameaças críticas, as vulnerabilidades de baixa severidade funcionam como indicadores de maturidade do processo de desenvolvimento e contribuem para o aprimoramento das práticas internas de segurança.

A análise SAST demonstrou que, embora o código do PoP-ERP não apresente grande quantidade de vulnerabilidades críticas, havia problemas pontuais capazes de comprometer a confidencialidade e a integridade dos dados trafegados e armazenados.

5.3 Avaliação da Análise de SBOM (Trivy)

A análise SCA/SBOM utilizou o Trivy em modo de varredura de sistema de arquivos, conforme abaixo:

```
trivy fs --exit-code 1 --severity HIGH,CRITICAL .
trivy fs --format json --output trivy-fs-report.json .
```

A ferramenta avaliou tanto dependências Python quanto pacotes do sistema operacional.

5.3.1 Resultados da Análise SBOM

Tabela 5.3: Vulnerabilidades encontradas na análise SBOM

Biblioteca	**Versão**	**CVE**	**Severidade**	**Versão corrigida**
cryptography	39.0.0	CVE-2023-0286	Alta	39.0.1
cryptography	39.0.0	CVE-2023-50782	Alta	42.0.0
cryptography	39.0.0	CVE-2024-26130	Alta	42.0.4
Gunicorn	20.1.0	CVE-2024-1135	Alta	22.0.0
Gunicorn	20.1.0	CVE-2024-6827	Alta	23.0.0
Total de vulnerabilidades	–	**5**	–	–

5.3.2 Discussão e Ações Corretivas SBOM

A análise da cadeia de suprimentos evidenciou que as vulnerabilidades encontradas estavam concentradas exclusivamente nas bibliotecas cryptography e gunicorn, ambas amplamente utilizadas no ecossistema Python. A biblioteca cryptography, na versão 39.0.0, apresentou três vulnerabilidades classificadas como de alta severidade (CVE-2023-0286, CVE-2023-50782 e CVE-2024-26130), todas corrigidas mediante atualização para versões superiores recomendadas pelo relatório, como 39.0.1, 42.0.0 e 42.0.4. De forma semelhante, o web server gunicorn, na versão 20.1.0, apresentou duas vulnerabilidades críticas relacionadas a ataques de HTTP Request Smuggling, solucionadas com a migração para as versões 22.0.0 e 23.0.0. Dessa forma, a mitigação das vulnerabilidades identificadas ocorreu integralmente por meio da atualização das dependências no arquivo requirements.txt, não havendo evidências de falhas relacionadas a frameworks ou bibliotecas adicionais. A correção refletiu diretamente na eliminação de todos os riscos reportados pelo Trivy, resultando em uma cadeia de suprimentos mais robusta e alinhada às práticas modernas de segurança.

Essas intervenções foram acompanhadas da fixação explícita das versões no arquivo `requirements.txt`, estratégia que reduz a variação não controlada das dependências ao longo do tempo (*dependency drift*). Ao estabelecer versões exatamente definidas, evita-se que novas instalações do ambiente passem a utilizar releases mais recentes — e potencialmente vulneráveis ou incompatíveis — das bibliotecas, garantindo maior estabilidade, reprodutibilidade e previsibilidade durante a evolução do projeto. A análise SBOM demonstrou, portanto, a importância da atualização contínua e do monitoramento sistemático dos componentes externos.

5.4 Avaliação do Teste Dinâmico (DAST)

A avaliação dinâmica foi conduzida utilizando o OWASP ZAP em modo *Baseline*, direcionado à instância de *staging* do sistema PoP-ERP. Essa abordagem executa exclusivamente verificações passivas, isto é, não envia cargas de ataque nem altera o estado da aplicação, limitando-se a analisar as respostas HTTP em busca de padrões conhecidos de risco. Dessa forma, os resultados refletem problemas de configuração, políticas de segurança insuficientes e comportamentos perigosos observáveis durante a navegação comum.

5.4.1 Resultados Obtidos

O relatório gerado registrou um total de **16 alertas** classificados como *WARN-NEW*, **nenhum** alerta crítico (*FAIL-NEW*) e **54 verificações bem-sucedidas** marcadas como *PASS*. As vulnerabilidades encontradas concentram-se principalmente em três grandes eixos: ausência de cabeçalhos de segurança essenciais, cookies configurados sem atributos de proteção e possibilidade de manipulação de atributos HTML que podem, em cenários específicos, favorecer ataques de *Cross-Site Scripting* (XSS).

A Tabela 5.4 consolida os resultados de forma agrupada.

Tabela 5.4: Resumo das vulnerabilidades detectadas pelo OWASP ZAP (Baseline)

Categoria	**Qtde.**	**Risco**	**Descrição resumida**
Cabeçalhos de segurança ausentes	5	Alto	Ausência de HSTS, CSP, Permissions-Policy, X-Content-Type-Options e SRI.
Cookies sem atributos de segurança	6	Alto	Cookies sem `Secure`, `HttpOnly` ou ambos, aumentando o risco de sequestro de sessão.
Riscos de XSS (atributos controláveis pelo usuário)	3	Médio	Possibilidade de manipulação de atributos HTML que podem levar a XSS refletido.
Exposição menor de informações	2	Baixo	Comentários suspeitos e trechos em Base64 que podem revelar metadados internos.
Total	**16**	–	–

5.4.2 Discussão e Ações Corretivas DAST

A análise dinâmica destacou vulnerabilidades que não podem ser detectadas por ferramentas estáticas ou por meio da inspeção da cadeia de suprimentos (SBOM). A seguir, discute-se cada categoria de alerta à luz de seu impacto e das correções recomendadas.

1. Ausência de cabeçalhos de segurança (Severidade alta)

Foram detectadas ausências importantes de cabeçalhos como:

- **Strict-Transport-Security (HSTS)**: essencial para evitar ataques de *downgrade* de protocolo;
- **Content-Security-Policy (CSP)**: fundamental para limitar fontes de scripts e prevenir ataques de XSS;
- **Permissions-Policy**: controla o acesso a APIs como câmera, microfone e geolocalização;
- **X-Content-Type-Options**: previne ataques de *MIME sniffing*; e
- **Subresource Integrity (SRI)**: garante a integridade de arquivos carregados via CDN.

A ausência desses cabeçalhos reduz significativamente a resiliência da aplicação contra ataques que exploram manipulação de conteúdo, injeção de scripts e comportamento indevido do navegador.

Ação corretiva: Configurações adicionais no servidor Nginx e no *middleware* do Django devem ser aplicadas para definir e reforçar tais cabeçalhos, conforme recomendações do projeto OWASP Secure Headers.

2. Cookies sem atributos de segurança (Severidade alta)

O ZAP identificou múltiplos cookies sem os atributos `Secure` e `HttpOnly`. Tais configurações permitem que:

- cookies de sessão possam ser acessados por scripts maliciosos, em razão da ausência do atributo `HttpOnly`;
- cookies sejam transmitidos por conexões inseguras, devido à ausência do atributo `Secure`.

Esses problemas aumentam o risco de sequestro de sessão (*session hijacking*) e exposição indevida de credenciais.

Ação corretiva: definir no Django:

```
SESSION_COOKIE_SECURE = True
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SAMESITE = 'Strict'
```

3. Possibilidade de XSS via atributos controláveis (Severidade média)

O alerta `10031` indica que alguns atributos HTML renderizados podem ser influenciados pela entrada do usuário, criando um vetor de risco para XSS caso outras validações falhem ou caso scripts externos sejam introduzidos.

Embora o ZAP Baseline não execute cargas de ataque, este alerta sugere pontos que merecem atenção no *template engine* do Django.

Ação corretiva:

- revisar as variáveis inseridas nos templates;
- reforçar o uso de `{{ var|escape }}`;
- adicionar validações de entrada no backend.

4. Exposição menor de informações (Baixa severidade)

Foram detectados dois tipos de vazamentos informacionais:

- comentários suspeitos em arquivos JavaScript (`10027`);
- conteúdos codificados em Base64 em páginas autenticadas (`10094`).

Embora não representem risco direto, esses elementos podem revelar detalhes internos da aplicação ou informações úteis a atacantes.

Ação corretiva: remoção de comentários, minificação de arquivos estáticos e revisão de trechos codificados.

5.4.3 Síntese Geral da Ferramenta OWASP ZAP

Apesar da ausência de falhas críticas, os **16 alertas** identificados revelam oportunidades claras de fortalecimento das políticas de segurança do PoP-ERP. Particularmente, os problemas relacionados a cabeçalhos de segurança e configuração de *cookies* representam vetores relevantes de ataque e devem ser tratados como prioridade. As ações corretivas discutidas alinham a aplicação a boas práticas amplamente recomendadas pela OWASP e contribuem para elevar significativamente a resiliência da aplicação contra ataques baseados em navegador.

5.5 Síntese Integrada da Avaliação

Tabela 5.5: Comparativo entre SAST, SBOM e DAST

Método	**Vulnerabilidades**	**Origem**	**Correção**
SAST (Bandit)	54	Código-fonte	Refatoração
SBOM (Trivy)	5	Dependências	Atualizações
DAST (ZAP)	16	Configuração/execução	Políticas e *patches*
Total	**75**	–	–

A integração dos três métodos proporcionou uma visão holística da segurança do PoP-ERP. A análise estática permitiu identificar anomalias estruturais de código; a análise SBOM expôs riscos herdados de componentes de terceiros; e a análise dinâmica revelou fragilidades que emergem somente durante a execução da aplicação. Em conjunto, esses resultados forneceram um mapeamento amplo da superfície de ataque e orientaram correções eficazes.

Capítulo 6

Conclusão

Este capítulo apresenta as conclusões gerais deste trabalho, sintetizando os resultados alcançados, as dificuldades enfrentadas ao longo do desenvolvimento, as implicações organizacionais da adoção de um pipeline de segurança e as possibilidades para trabalhos futuros. As análises conducentes ao desenvolvimento desta solução foram descritas nos Capítulos 4 e 5, que demonstraram, em detalhe, a viabilidade e a eficácia da integração de mecanismos de segurança no ciclo de desenvolvimento do PoP-ERP.

6.1 Síntese dos Resultados

A implementação do pipeline de segurança proposto neste trabalho permitiu identificar e mitigar vulnerabilidades em diferentes camadas do sistema. Conforme detalhado no Capítulo 5, a utilização combinada de técnicas de *Static Application Security Testing* (SAST), *Software Composition Analysis* (SBOM/SCA) e *Dynamic Application Security Testing* (DAST) revelou fragilidades tanto no código-fonte quanto nas dependências e configurações do ambiente de execução.

Os resultados evidenciaram vulnerabilidades críticas, como a desativação da verificação de certificados TLS, permissões excessivamente liberais em diretórios e dependências desatualizadas, além de fragilidades dinâmicas relacionadas a cabeçalhos de segurança e atributos de sessão. A correção sistemática dessas falhas elevou significativamente o nível de maturidade do PoP-ERP, reforçando a confiabilidade do sistema e aprimorando a segurança operacional.

6.2 Dificuldades Técnicas e Organizacionais

O processo de construção e consolidação da pipeline enfrentou desafios que transcenderam questões meramente técnicas. Do ponto de vista organizacional, uma das principais dificuldades consistiu em introduzir novas políticas de segurança em um ambiente já estabelecido, no qual práticas consolidadas e rotinas operacionais tendem a predominar sobre iniciativas de melhoria contínua. A apresentação dos resultados para níveis hierárquicos superiores exigiu clareza, objetividade e evidências quantitativas capazes de demonstrar os riscos identificados e o potencial impacto para o ecossistema institucional.

Outro obstáculo relevante consistiu na necessidade de sensibilizar os desenvolvedores sobre a importância de considerar os resultados das ferramentas de segurança como parte integrante do processo de desenvolvimento. Em muitos casos, a percepção inicial era de que as análises prejudicariam prazos ou introduziriam complexidade adicional. No entanto, à medida que os resultados demonstraram benefícios concretos, ficou evidente que a refatoração guiada pelas ferramentas não apenas reduziu vulnerabilidades, mas também contribuiu para a melhoria estrutural do código, reforçando boas práticas e favorecendo a manutenção evolutiva do sistema.

6.3 Impacto na Cultura de Desenvolvimento

A implantação do pipeline representou um passo significativo rumo à institucionalização de práticas de *DevSecOps*. A automatização da análise de segurança reduziu a dependência de verificações manuais e introduziu um fluxo contínuo de monitoramento de riscos. Essa mudança contribuiu para a formação de uma cultura organizacional mais orientada à prevenção, promovendo a conscientização de equipes de desenvolvimento e de gestão sobre a importância da segurança como componente indissociável do ciclo de vida do software.

A consolidação dessa cultura depende, entretanto, de esforços contínuos. O acompanhamento periódico das análises, o cumprimento das correções identificadas pelo pipeline e o alinhamento entre desenvolvedores, analistas de segurança e gestores constituem elementos fundamentais para a manutenção e evolução da solução implantada.

6.4 Trabalhos Futuros

Diversas oportunidades de aprimoramento foram identificadas ao longo do desenvolvimento desta pesquisa. Entre elas, destaca-se a ampliação do pipeline para abranger mecanismos de verificação de autenticidade, como assinaturas digitais de dependências e imagens de contêiner, permitindo a implementação de um modelo mais robusto de *source of truth* e aumentando a confiabilidade dos artefatos utilizados durante o desenvolvimento e a implantação.

Outra direção promissora envolve o uso avançado do OWASP ZAP, configurado para ataques mais severos, específicos e customizados, incluindo técnicas de fuzzing de maior profundidade e perfis de exploração adaptados ao comportamento da aplicação. Da mesma forma, a integração de ferramentas de **IAST (*Interactive Application Security Testing*)**, que realizam testes de segurança interativos durante a execução da aplicação, permitiria detectar vulnerabilidades que somente se manifestam em cenários reais de uso, complementando as abordagens SAST e DAST tradicionalmente empregadas. Esse tipo de teste combina análise dinâmica e monitoramento interno do fluxo de execução, possibilitando identificar falhas lógicas, problemas de validação e violações de segurança em tempo real.

Por fim, recomenda-se a adoção de métricas contínuas de segurança, alinhadas ao conceito de *Security Scorecards*, permitindo o acompanhamento histórico do nível de maturidade e a identificação de regressões ao longo do tempo.

6.5 Considerações Finais

Conclui-se que a pipeline de segurança desenvolvida neste trabalho demonstrou grande efetividade na identificação precoce de vulnerabilidades e no fortalecimento da proteção do PoP-ERP. A integração dos mecanismos propostos contribuiu para a criação de um ciclo contínuo de melhoria, alinhado às melhores práticas de desenvolvimento seguro. Espera-se que a iniciativa aqui apresentada sirva como base para futuras evoluções e incentive a adoção de políticas de segurança cada vez mais maduras no ambiente institucional.

Referências Bibliográficas

APACHE SOFTWARE FOUNDATION. *Log4j – Apache Logging Services*. 2025. Acesso em: 25 nov. 2025. Disponível em: <https://logging.apache.org/log4j/2.x/>.

AQUA SECURITY SOFTWARE LTD. *Aqua Security - Cloud Native Security Platform*. 2025. Acesso em: 25 nov. 2025. Disponível em: <https://www.aquasec.com/>.

BOYENS, J. et al. *Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations*. [S.l.], 2022. Atualizado em 2024 (versão r1-upd1 disponível). Disponível em: <https://doi.org/10.6028/NIST.SP.800-161r1>.

CAVEN, P. J.; ABBOTT, J.; CAMP, L. J. Towards a More Secure Ecosystem: Implications for Cybersecurity Labels and SBOMs. *SSRN Preprint*, 2023. Preprint.

CHAUHAN, M.; SHIAELES, S. An Analysis of Cloud Security Frameworks, Problems and Proposed Solutions. *Network*, v. 3, p. 422–450, 2023.

CHAVA, A. CI/CD and Automation in DevOps Engineering. *Asian Journal of Research in Computer Science*, v. 17, p. 73–80, 2024.

CORPORATION, O. *Oracle Security Alert – CVE-2025-61882*. 2025. <https://www.oracle.com/security-alerts/alert-cve-2025-61882.html>. Acesso em: 25 nov. 2025.

CRUZ, D. B.; ALMEIDA, J. R.; OLIVEIRA, J. L. Open Source Solutions for Vulnerability Assessment: A Comparative Analysis. *IEEE Access*, v. 11, p. 100234–100255, 2023.

CYBER, Q. Cve-2025-61882 remote code execution in oracle e-business suite (ebs). *Quorum Cyber Threat Intelligence*, 2025. Acesso via web – monitoramento ativo da Cl0p.

DJANGO SOFTWARE FOUNDATION. *Django Documentation: Security Overview*. [S.l.], 2023. Acessado em 2025. Disponível em: <https://docs.djangoproject.com/en/4.2/topics/security/>.

DOCKER INC. *Docker - Build, Ship, and Run Any App, Anywhere*. 2025. Acesso em: 25 nov. 2025. Disponível em: <https://www.docker.com/>.

FADLIL, A.; RIADI, I.; MU'MIN, M. A. Mitigation from SQL Injection Attacks on Web Server using Open Web Application Security Project Framework. *International Journal of Engineering, Transactions A: Basics*, v. 37, p. 635–645, 2024.

FONSECA, J.; VIEIRA, M.; MADEIRA, H. The role of dynamic security testing in web application protection. *Journal of Information Security*, v. 11, n. 3, p. 121–135, 2020.

FOUNDATION, O. *OpenStack: The Most Widely Deployed Open Source Cloud Software in the World*. 2025. Acesso em: 25 nov. 2025. Disponível em: <https://www.openstack.org/>.

GITHUB INC. *GitHub - Where the world builds software*. 2025. Acesso em: 25 nov. 2025. Disponível em: <https://github.com/>.

GITLAB INC. *GitLab - The DevSecOps Platform*. 2025. Acesso em: 25 nov. 2025. Disponível em: <https://about.gitlab.com/>.

GUDURU, S. DevSecOps Automation: SAST/DAST Integration in GitLab CI/CD with Semgrep, OWASP ZAP, and Dependency-Check. *International Journal of Science and Research (IJSR)*, v. 9, p. 1893–1898, 2020.

GUPTA, S.; GUPTA, B. B. Enhanced XSS Defensive Framework for Web Applications Deployed in the Virtual Machines of Cloud Computing Environment. *Procedia Technology*, v. 24, p. 1595–1602, 2016.

H.K., K. K. et al. An Approach to basic GUI-enabled CI/CD pipeline with Static Analysis tool. *Journal of University of Shanghai for Science and Technology*, v. 23, p. 683–691, 2021. ISSN 1007-6735.

INC., A. *Grype: Vulnerability Scanner for SBOM and Container Images*. 2024. Acesso em: 25 nov. 2025. Disponível em: <https://github.com/anchore/grype>.

INC., A. *Syft: SBOM Generator*. 2024. Acesso em: 25 nov. 2025. Disponível em: <https://github.com/anchore/syft>.

INTELLIGENCE, C. Campaign targeting oracle e-business suite via zero-day vulnerability (cve-2025-61882). *CrowdStrike Blog*, 2025. Avaliação de ameaça e IOCs publicados. Disponível em: <https://www.crowdstrike.com/en-us/blog/crowdstrike-identifies-campaign-targeting-oracle-e-business-suite-zero-day-CVE-2025-61882/>.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION. *ISO/IEC 5230:2020 — Information technology — OpenChain Specification*. 2020. Acesso em: 25 nov. 2025. Disponível em: <https://www.iso.org/standard/81877.html>.

INVICTI SECURITY LTD. *Acunetix - Web Application Security Scanner*. 2025. Acesso em: 25 nov. 2025. Disponível em: <https://www.acunetix.com/>.

KOGA, H. H. *Um estudo sobre a aplicação de ferramentas para a proteção de cadeias de suprimento de software*. 48 p. Monografia (Graduação em Engenharia da Computação), 2025. Orientador: Dr. Eduardo de Lucena Falcão. Departamento de Engenharia de Computação e Automação, Centro de Tecnologia.

KONERU, N. M. K. Integrating Security into CI/CD Pipelines: A DevSecOps Approach with SAST, DAST, and SCA Tools. *International Journal of Science and Research Archive*, v. 3, p. 250–265, 2021.

LAUDON, K. C.; LAUDON, J. P. *Management Information Systems: Managing the Digital Firm*. 16. ed. New Jersey: Pearson, 2020.

LEW, K. et al. Distributed Software Build Assurance for Software Supply Chain Integrity. *Applied Sciences*, v. 14, p. 9262, 2024.

MALIPEDDI, A. K.; PASUNURU, S. Securing DevOps CI/CD pipelines with Agent-Based and Agentless Solutions. *International Scientific Journal of Engineering and Management*, v. 2, p. 1–5, 2023.

MOMJIAN, B. *PostgreSQL Documentation*. [S.l.], 2022. Acesso em: 4 dez. 2025. Disponível em: <https://www.postgresql.org/docs/>.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. *NIST - National Institute of Standards and Technology*. 2025. Acesso em: 25 nov. 2025. Disponível em: <https://www.nist.gov/>.

O'BRIEN, J. A.; MARAKAS, G. M. *Management Information Systems*. 10. ed. New York: McGraw-Hill, 2011.

OPENSTACK FOUNDATION. *Bandit: Python Security Linter*. 2024. Acesso em: 25 nov. 2025. Disponível em: <https://bandit.readthedocs.io/>.

OWASP. *OWASP Web Security Testing Guide*. 2023. <https://owasp.org/www-project-web-security-testing-guide/>. Acesso em: 20 jan. 2025.

OWASP Foundation. *Zed Attack Proxy (ZAP) – Official Documentation*. 2023. <https://www.zaproxy.org/docs/>. Acesso em: 20 jan. 2025.

OWASP FOUNDATION. *CycloneDX - Software Bill of Materials (SBOM) Standard*. 2025. Acesso em: 25 nov. 2025. Disponível em: <https://cyclonedx.org/>.

PONAKA, K. R. Shift-left approach for Vulnerability Management in SDLC. *International Journal of Scientific Research in Engineering and Management (IJSREM)*, v. 5, p. 1–5, 2021.

PONAKA, K. R. Automated Security Vulnerability Backlog Management. *International Journal of Scientific Research in Engineering and Management (IJSREM)*, v. 6, p. 1–4, 2022.

PORTSWIGGER LTD. *Burp Suite Professional - The World's Leading Toolkit for Web Application Security Testing*. 2025. Acesso em: 25 nov. 2025. Disponível em: <https://portswigger.net/burp/pro>.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL: The World's Most Advanced Open Source Database*. 2025. Acesso em: 25 nov. 2025. Disponível em: <https://www.postgresql.org/>.

PYTHON SOFTWARE FOUNDATION. *Python Programming Language – Official Website*. 2025. Acesso em: 25 nov. 2025. Disponível em: <https://www.python.org/>.

R2C, I. *Semgrep: Lightweight Static Analysis*. 2024. Acesso em: 25 nov. 2025. Disponível em: <https://semgrep.dev>.

RESCORLA, E. *The Transport Layer Security (TLS) Protocol Version 1.3*. [S.l.], 2018. (Request for Comments, 8446). Disponível em: <https://www.rfc-editor.org/info/rfc8446>. Disponível em: <https://www.rfc-editor.org/info/rfc8446>.

ROSSUM, G. V.; DRAKE, F. L. *Python Tutorial*. Release 3.11. Python Software Foundation, 2023. Acessado em 2025. Disponível em: <https://docs.python.org/3/tutorial/>.

SECURITY, A. *Trivy: Vulnerability and Compliance Scanner*. 2024. Acesso em: 25 nov. 2025. Disponível em: <https://github.com/aquasecurity/trivy>.

SETH, A. et al. Comparing effectiveness and efficiency of Interactive Application Security Testing (IAST) and Runtime Application Self-Protection (RASP) tools in a large java-based system. *Empirical Software Engineering*, v. 30, 2025.

SHAH, A.; PATEL, R.; SINGH, K. Dynamic application security testing: A comprehensive review. In: *Proceedings of the International Conference on Cybersecurity*. [S.l.]: IEEE, 2021. p. 45–52.

SHEN, M. et al. An empirical study on the use of static analysis tools in open source embedded software. *Empirical Software Engineering Journal*, 2023.

SIAVVAS, M. et al. Static analysis-based approaches for secure software development. In: *Security in Computer and Information Sciences (Euro-CYBERSEC 2018)*. [S.l.]: Springer, Cham, 2018.

SINGH, N. et al. CI/CD Pipeline for Web Applications. *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, v. 11, p. 5218–5226, 2023.

SONARSOURCE. *SonarQube Documentation*. 2024. Acesso em: 25 nov. 2025. Disponível em: <https://docs.sonarsource.com/sonarqube>.

TADHANI, J. R. et al. Securing web applications against XSS and SQLi attacks using a novel deep learning approach. *Scientific Reports*, v. 14, p. 1803, 2024.

TORRES-ARIAS, S.; GEER, D.; MEYERS, J. S. A Knowing Viewpoint on Software Bill of Materials Quality When You See It. *IEEE Security & Privacy*, IEEE Computer and Reliability Societies, v. 21, p. 50–54, 2023.

TUDELA, F. M. et al. On Combining Static, Dynamic and Interactive Analysis Security Testing Tools to Improve OWASP Top Ten Security Vulnerability Detection in Web Applications. *Applied Sciences*, v. 10, p. 9119, 2020.

UKHANOV, P. et al. Oracle e-business suite zero-day exploited in widespread extortion campaign. *Google Cloud Blog*, 2025. Análise conjunta com Mandiant. Disponível em: <https://cloud.google.com/blog/topics/threat-intelligence/oracle-ebusiness-suite-zero-day-exploitation>.

WADHAMS, Z.; IZURIETA, C.; REINHOLD, A. M. Barriers to using static application security testing (sast) tools: A literature review. In: *Workshop on Human-Centric Software Engineering & Cyber Security (HCSE&CS-2024)*. [S.l.]: IEEE/ACM, 2024.

YIMER, S.; GIZACHEW, B. The development of a web-based application security testing framework in Addis Ababa, Ethiopia. *Global Journal of Computer Sciences: Theory and Research*, v. 12, p. 12–22, 2022.

Apêndice A

Informações adicionais

Tabela A.1: Tabela completa da Pesquisa Bibliográfica

Ref.	**Título**	**Autores/Ano**	**Objetivo**	**Principais Contribuições/Resultados**
1	Um Estudo sobre a Aplicação de Ferramenta S...	Koga (2025) (**??**)	Aumentar segurança em bibliotecas de terceiros via SBOM.	A pipeline se mostrou robusta ao identificar problemas em bibliotecas desatualizadas e ao cumprir seu papel em auxiliar a equipe de desenvolvimento na atualização das mesmas.
2	A Knowing Viewpoint on Software Bill of Materials Quality When You See It	Torres-Arias et al. (2023) (TORRES-ARIAS; GEER; MEYERS, 2023)	Avaliar qualidade estrutural de SBOMs.	Apenas 1% dos SBOMs contêm os elementos mínimos NTIA para todos os componentes; A maioria fornece metadados ricos, mas incompletos; Diferenças no número de componentes reportados entre ferramentas em mais de 16% das amostras, indicando discrepâncias na detecção de pacotes.
3	CI/CD and Automation in DevOps Engineering	Chava (2024) (CHAVA, 2024)	Revisar boas práticas de CI/CD.	Acelera ciclo; reduz erros; discute desafios de adoção.
4	CI/CD Pipeline for Web Applications	Singh et al. (2023) (SINGH et al., 2023)	Descrever pipeline CI/CD web.	Reduz TTM; automação melhora qualidade; desafios de infraestrutura.

Continua na próxima página

Continuação da Tabela A.1

Ref.	Título	Autores/Ano	Objetivo	Principais Contribuições/Resultados
5	Basic GUI-enabled CI/CD Pipeline with Static Analysis Tool	Kumar et al. (2021) (H.K. et al., 2021)	Criar pipeline com análise estática.	Pipeline funcional com GUI; limitações de suporte.
6	Securing CI/CD Pipelines with Agent-Based and Agentless Solutions	Malipeddi & Pasunuru (2023) (MALIPEDDI; PASUNURU, 2023)	Comparar gerenciamento de segredos.	Agent-based = granularidade; Agentless = escalabilidade.
7	Distributed Software Build Assurance for Supply Chain Integrity	Lew et al. (2024) (LEW et al., 2024)	Modelo distribuído p/ verificar builds.	Verificação 2–3x mais rápida; limitações de reprodutibilidade.
8	Comparing Effectiveness of IAST and RASP Tools	Seth et al. (2025) (SETH et al., 2025)	Comparar IAST/RASP.	IAST eficaz; RASP limitado; análise em OpenMRS.
9	Combining Static, Dynamic and Interactive Security Testing Tools	Tudela et al. (2020) (TUDELA et al., 2020)	Integrar SAST/DAST/IAST.	Aumenta detecção; exige auditoria manual.
10	Open Source Solutions for Vulnerability Assessment	Cruz et al. (2023) (CRUZ; ALMEIDA; OLIVEIRA, 2023)	Comparar ferramentas SAST/DAST/SCA.	Cria baseline comparativa.
11	Integrating Security into CI/CD Pipelines	Koneru (2021) (KONERU, 2021)	Implementar DevSecOps.	Pipeline integrado; recomendações práticas.
12	Automated Security Vulnerability Backlog Management	Ponaka (2022) (PONAKA, 2022)	Automatizar backlog.	Propõe automação com SAST/DAST/SCA.
13	DevSecOps Automation in GitLab CI/CD	Guduru (2020) (GUDURU, 2020)	Integrar SAST/DAST/SBOM no GitLab.	Semgrep + ZAP + Dependency-Check; SLSA scorecards.

Continua na próxima página

Continuação da Tabela A.1

Ref.	**Título**	**Autores/Ano**	**Objetivo**	**Principais Contribuições/Resultados**
14	Cybersecurity Labels and SBOMs: Implications	Caven et al. (2023) (CAVEN; ABBOTT; CAMP, 2023)	Analisar rótulos e SBOM.	Aumenta transparência e conformidade.
15	Web-based Security Testing Framework	Yimer & Gizachew (2022) (YIMER; GIZACHEW, 2022)	Testes de segurança web.	Mapeia desafios locais; propõe framework.
16	Mitigation from SQL Injection Attacks using OWASP Framework	Fadlil et al. (2024) (FADLIL; RIADI; MU'MIN, 2024)	Mitigar SQLi com OWASP.	Testes ZAP e SQLMap; valida eficácia.
17	Analysis of Cloud Security Frameworks	Chauhan & Shiaeles (2023) (CHAUHAN; SHIAELES, 2023)	Comparar frameworks de segurança.	Identifica lacunas e diferenças.
18	Deep Learning Detection of XSS and SQLi (CNN-LSTM)	Tadhani et al. (2024) (TADHANI et al., 2024)	Detectar ataques via DL.	Modelo CNN-LSTM com alta acurácia.
19	Enhanced XSS Defensive Framework for VMs	Gupta & Gupta (2016) (GUPTA; GUPTA, 2016)	Defesa contra XSS.	Inspeção de scripts em ambientes virtualizados.
20	Shift-left Approach for Vulnerability Management	Ponaka (2021) (PONAKA, 2021)	Integrar gating em CI/CD.	Bloqueia builds vulneráveis.